EINDHOVEN UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering
Control Systems Technology

AI-Enabled Systems Engineering: A Case Study in Failure Mode and Effects Analysis

Master's Thesis

S.S. Moosavi Lar

Supervisor:
Dr. Pascal Etman

Draft version

Eindhoven, May 2026

Abstract

Digital Engineering aims to move engineering practice from document-centred information exchange toward connected, model-based engineering environments. In this context, the usefulness of engineering models depends strongly on their integrity, including completeness, structure, traceability, and reviewability.

This thesis proposes a framework for improving model integrity in Digital Engineering environments. The framework combines SysML-oriented model parsing, ontology-based reasoning guidance, traceability support, and AI-assisted analysis. Artificial intelligence is not treated as the main research goal, but as one enabling method within a broader model integrity framework.

The framework is demonstrated using Failure Mode and Effects Analysis (FMEA) as a downstream case study task. Two SysML-style models are used: a simple flashlight model and a more complex quadcopter model. The parser extracts model elements such as parts, requirements, actions, ports, connections, and allocations. These outputs are then combined with a domain-agnostic FMEA ontology to support failure mode generation, review, scoring, explanation, and revision.

The results show that structured model information and semantic guidance can improve the traceability and reviewability of downstream engineering analysis. The work also highlights limitations, including dependence on model completeness and the continued need for human engineering judgement.

Preface

This thesis was inspired by a central question in modern engineering practice: how can engineers trust increasingly complex digital models when decisions, analyses, and risks depend on them? As engineering organisations move from document-based work toward model-based and digitally connected environments, the model is no longer only a representation of the system. It also becomes part of the decision-making environment itself. This observation motivated the present research on model integrity in Digital Engineering environments.

The project also emerged from a broader interest in systems thinking. Engineering work often involves connecting different worlds: abstract requirements and physical behaviour, human judgement and automated reasoning, established practice and emerging technology. Digital Engineering promises to connect these worlds through linked models, simulations, data, and analyses. However, that promise can only be realised if the links are trustworthy. This thesis therefore approaches artificial intelligence not as a replacement for systems engineering judgement, but as one possible assistant within a disciplined structure of evidence, semantics, and traceability.

I would like to express my sincere gratitude to my supervisor, Dr. Pascal Etman, for his guidance, feedback, and support throughout the development of this research. His supervision helped shape the project into a clearer and more academically rigorous investigation. I am also grateful to the Department of Electrical Engineering and the Artificial Intelligence and Engineering Systems programme at Eindhoven University of Technology for providing the academic environment in which this work could be developed.

This research journey involved several shifts in focus. It began with a broad interest in Digital Engineering, Model-Based Systems Engineering, and the connection between descriptive and analytical models. Over time, the work became more focused on model integrity, semantic grounding, traceability, and the responsible use of AI-supported reasoning for engineering analysis. The final thesis reflects that journey by combining systems engineering foundations, semantic technologies, traceability principles, and AI-assisted reasoning into a framework for examining engineering models and supporting traceable Failure Mode and Effects Analysis outputs.

Writing this thesis has reinforced the importance of disciplined engineering thinking in the age of AI. The central lesson is simple but important: powerful AI tools become useful in engineering only when they are connected to structure, evidence, and accountability. Without those elements, automation may produce fluent text but weak engineering knowledge. With them, AI can become a meaningful assistant in maintaining and improving the integrity of digital engineering models.

Contents

List of Figures

List of Tables

Listings

Chapter 1

Introduction

1.1 Background and Motivation

Modern engineering systems are increasingly complex. They often combine mechanical, electrical, software, control, safety, operational, and human factors considerations within a single system. As a result, engineering decisions are rarely based on isolated pieces of information. Instead, they depend on many connected artefacts that describe what the system should do, how it is structured, how it behaves, how it is analysed, and how it is verified.

These artefacts include requirements, functions, components, parameters, assumptions, simulations, verification evidence, and operational constraints. Each artefact may represent only one part of the engineering problem, but decisions usually require understanding how these parts relate to each other. For example, a design parameter may affect a simulation result, a simulation result may support a verification claim, and a verification claim may be linked to one or more system requirements. In this sense, the value of engineering information depends not only on the information itself, but also on the quality of the relationships between pieces of information.

Traditional document-centric engineering practices have difficulty managing these relationships. Engineering information is often distributed across reports, spreadsheets, diagrams, models, emails, databases, and tool-specific files. Although these artefacts may contain useful information, their relationships are often difficult to maintain manually. As projects grow in scale and complexity, it becomes increasingly challenging to keep information consistent, reusable, and available for decision-making.

Digital Engineering has emerged as a response to this challenge. It aims to support more connected, model-based, and lifecycle-oriented engineering practice. Instead of treating engineering artefacts as separate documents, Digital Engineering seeks to create digital environments in which models, data, analyses, and evidence can be connected and reused throughout the system lifecycle [35, 33]. The purpose is not only to digitise existing documents, but to improve how engineering information is represented, related, updated, and used.

Model-Based Systems Engineering (MBSE) provides an important foundation for this transition. MBSE shifts systems engineering from document-centred communication toward model-centred representation and reasoning. System models can describe requirements, functions, structures, behaviours, interfaces, constraints, and verification relationships in a more explicit way than traditional documents [25]. This makes MBSE a key enabler for Digital Engineering, because it provides structured model-based artefacts that can support lifecycle decision-making.

These developments create the background for studying how engineering information can be represented, connected, updated, and reused in Digital Engineering environments.

As engineering practice becomes more model-based, the quality of the model ecosystem becomes increasingly important. The central concern is not only whether models exist, but whether the information within and between those models can reliably support engineering analysis and decision-making.

1.2 Problem Statement

Although Digital Engineering and MBSE aim to improve the connection of engineering information, the existence of models does not guarantee that the information is reliable, current, or usable for analysis. A model-based environment may contain many useful models and datasets, but the relationships between them may still be incomplete, ambiguous, outdated, or difficult to inspect. This creates a gap between the ambition of Digital Engineering and the practical use of model-based information for engineering reasoning.

In practice, requirements may not be linked clearly to functions. Assumptions may remain implicit rather than being represented as explicit model elements. Analysis results may become stale when the model changes. Parameters used in calculations may not be traceable to design variables. Different tools may use inconsistent terminology for the same concept, or the same term may be used with different meanings in different parts of the engineering environment. These issues make it difficult to determine whether the available information is still valid and whether a conclusion is supported by sufficient evidence.

These weaknesses reduce model integrity. In this thesis, model integrity is understood in practical terms as the reliability, traceability, currency, consistency, and usability of model information for downstream engineering analysis. A model ecosystem with low integrity may still appear complete at the surface level, but it may not support dependable reasoning. Engineers may have to manually inspect separate artefacts, reconstruct missing links, compare versions, interpret assumptions, and check whether the information used in an analysis is still valid.

Poor model integrity therefore makes engineering analysis more difficult to perform, justify, and maintain. Analysis tasks become slow because engineers must spend time searching for relevant information and reconstructing the reasoning behind previous decisions. They also become time-dependent because the usefulness of an analysis can decrease when the underlying model information changes. If a requirement, parameter, assumption, or analysis result is updated, previous conclusions may become incomplete, inconsistent, or no longer valid.

This issue affects several downstream engineering tasks, including failure analysis, risk assessment, verification planning, design review, trade-off analysis, compliance checking, and impact analysis. These tasks depend on knowing which requirements, functions, components, parameters, assumptions, and evidence are connected. When those connections are missing or unreliable, engineers must reconstruct the reasoning manually. This increases effort, slows down decision-making, and makes the analysis harder to update after model changes.

Therefore, the central problem addressed in this thesis is how poor model integrity limits the reliability, efficiency, traceability, and maintainability of downstream engineering analyses in Digital Engineering environments. The thesis focuses on this problem by studying how existing tools and methods can be organised into a framework that supports higher model integrity.

1.3 Research Goal

The goal of this thesis is to develop a framework for improving model integrity in Digital Engineering environments and to demonstrate its application through a concrete downstream

engineering analysis task. The thesis focuses on how engineering information can remain reliable, connected, traceable, and reusable when models, assumptions, parameters, analyses, and evidence evolve over time.

To achieve this goal, the thesis investigates existing tools and methods that can support higher model integrity. These include descriptive modelling notations, semantic representations such as ontologies, and artificial intelligence. Descriptive modelling notations can help represent system elements and relationships explicitly. Semantic representations can help clarify meaning and support reuse across contexts. Artificial intelligence can support selected reasoning and information-processing tasks when used within a controlled engineering workflow.

These tools and methods are not studied as isolated or competing solutions. Instead, they are considered as complementary mechanisms that can address different aspects of model integrity. The proposed framework aims to show how engineering information can be structured, linked, checked, and maintained so that downstream analysis tasks remain trustworthy after model changes.

The goal is not to present one tool or technology as the complete solution. Rather, the thesis aims to explain how different approaches can be combined within a Digital Engineering workflow to support higher model integrity. This is important because model integrity is not a single-tool problem. It depends on how information is represented, how relationships are maintained, how evidence is linked, and how changes are reflected in downstream analyses.

Failure Mode and Effects Analysis (FMEA) is used as the demonstration task because it is strongly dependent on the quality, traceability, and maintainability of engineering information. FMEA is not treated as the main goal of the thesis, but as a case study for examining how model integrity affects downstream engineering analysis. This makes it possible to evaluate the proposed framework in a concrete setting while keeping the broader focus on model integrity.

The expected outcome is a practical framework that helps engineers reason about how to select and combine suitable methods for improving the reliability, traceability, and maintainability of model-based engineering information.

1.4 Research Questions

This thesis is guided by two research questions.

RQ1: How can existing tools and methods, including descriptive modelling notations, semantic representations, and artificial intelligence, be organised into a practical framework for improving model integrity, with Failure Mode and Effects Analysis used as the demonstration case?

RQ2: To what extent does the proposed framework support the reliability, traceability, and maintainability of downstream engineering analysis, and how can the framework be improved, evaluated through FMEA as a case study?

The first research question concerns the development of the framework. It focuses on how existing tools and methods can be selected, related, and organised to support model integrity in Digital Engineering environments. The second research question concerns evaluation. It examines whether the proposed framework supports downstream engineering analysis in a concrete case study and identifies how the framework can be improved.

Together, the research questions move from framework development to case-study evaluation. This structure supports the broader aim of the thesis: to understand how higher model integrity can be achieved in Digital Engineering environments and how this can improve the reliability and maintainability of downstream engineering analyses.

1.5 Scope

This thesis focuses on model integrity in Digital Engineering environments. It studies selected tools and methods that are relevant to improving the reliability, traceability, currency, consistency, and usability of model-based engineering information. These tools and methods include descriptive modelling notations, semantic representations, ontologies, graph-based representations, and artificial intelligence.

The thesis uses a compact engineering case study to demonstrate and evaluate the proposed framework. The case study is suitable because it includes components, functions, parameters, assumptions, analytical relations, and performance outputs. This makes it sufficiently rich to represent common model integrity challenges, while remaining small enough to inspect and explain within the scope of an MSc thesis.

The thesis does not aim to build a complete industrial Digital Engineering platform. It also does not claim to solve all interoperability problems across engineering tools. Instead, the focus is on developing and evaluating a framework that explains how different methods can be combined to support higher model integrity.

The thesis also does not claim to replace domain experts, safety engineers, or engineering judgement. The proposed framework is intended to support engineering reasoning, not automate responsibility. Human judgement remains necessary for interpreting results, validating assumptions, and making final engineering decisions.

FMEA is used as the demonstration task for studying how model integrity affects downstream engineering analysis. However, the model integrity problem addressed in this thesis is broader than FMEA. The same underlying issues also affect other downstream tasks, such as risk assessment, verification planning, design review, trade-off analysis, compliance checking, and impact analysis.

1.6 Contributions

This thesis makes five main contributions.

First, it defines model integrity in practical terms as the reliability, traceability, currency, consistency, and usability of model information for downstream engineering analysis. This definition supports the thesis by focusing on how model information is actually used in engineering workflows, rather than treating model quality as an abstract property.

Second, the thesis reviews and organises tools and methods that can support model integrity in Digital Engineering. These include descriptive modelling notations, semantic representations, ontologies, graph-based representations, and artificial intelligence. The contribution lies in examining how these methods can support model integrity when considered together.

Third, the thesis proposes a framework for achieving higher model integrity using a combination of modelling, semantic representation, evidence linking, and AI-supported methods. The framework provides a structured way to reason about how engineering information should be represented, connected, maintained, and used in downstream analysis.

Fourth, the thesis demonstrates how the proposed framework can support downstream engineering analysis through an FMEA case study. The case study is used to examine how model integrity influences the reliability, traceability, and maintainability of analysis outputs.

Fifth, the thesis provides guidance on how artificial intelligence can be responsibly positioned as one supporting method within a broader model integrity framework. Rather than treating AI as the main goal, the thesis considers AI as one possible enabler that must be combined with structured model information, semantic grounding, and engineering accountability.

1.7 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 reviews the literature on Digital Engineering, MBSE, SysML v2, semantic interoperability, ontologies, traceability, FMEA, and AI-supported engineering. This chapter establishes the theoretical and technical background needed to understand the model integrity problem and the methods considered in the proposed framework.

Chapter 3 presents the methodology. It describes the research design, selected tools and methods, proposed framework, case study, and evaluation approach. The chapter explains how the framework is constructed and how its usefulness is examined through the selected downstream engineering analysis task.

Chapter 4 applies and evaluates the proposed framework using the case study. It presents the results of the framework application and discusses how the framework supports reliability, traceability, and maintainability in downstream engineering analysis.

Chapter 5 concludes the thesis. It summarises the main findings, discusses the implications and limitations of the research, and proposes directions for future work.

Chapter 1 Draft Outline

This section are the sentences that later turned into paragraphs in this chapter.

1. **Background and Motivation**
   - **1.1.** Modern engineering systems are increasingly complex.
   - **1.2.** Engineering decisions depend on connected lifecycle information.
   - **1.3.** Information is spread across requirements, models, analyses, simulations, and evidence.
   - **1.4.** Document-centric practice makes these connections difficult to maintain.
   - **1.5.** Digital Engineering supports connected, model-based engineering practice.
   - **1.6.** MBSE provides structured models as a foundation for Digital Engineering.
   - **1.7.** This motivates studying how engineering information can be connected and reused.
2. **Problem Statement**
   - **2.1.** Models do not automatically make information reliable or usable.
   - **2.2.** Relationships between models, data, and evidence may be incomplete or outdated.
   - **2.3.** Weak links, implicit assumptions, and inconsistent terminology reduce model integrity.
   - **2.4.** Poor model integrity makes analysis slower and harder to justify.
   - **2.5.** Model changes can make previous analyses incomplete or invalid.
   - **2.6.** This affects failure analysis, risk assessment, verification, design review, and compliance.
   - **2.7.** The central problem is unreliable, inefficient, and hard-to-maintain downstream analysis.
3. **Research Goal**
   - **3.1.** Develop a framework for improving model integrity in Digital Engineering.
   - **3.2.** Study how engineering information can remain reliable, traceable, and reusable over time.
   - **3.3.** Investigate descriptive modelling notations, semantic representations, and artificial intelligence.
   - **3.4.** Treat these methods as complementary, not competing, approaches.
   - **3.5.** Show how information can be structured, linked, checked, and maintained.
   - **3.6.** Demonstrate the framework through Failure Mode and Effects Analysis (FMEA).
   - **3.7.** Use FMEA as a case study, not as the main goal of the thesis.
4. **Research Questions**
   - **4.1.** RQ1: How can existing tools and methods be organised into a framework for improving model integrity, with FMEA used as the demonstration case?
   - **4.2.** RQ2: To what extent does the framework support reliable, traceable, and maintainable downstream analysis, and how can it be improved in the case of FMEA?
5. **Scope**
   - **5.1.** Focus on model integrity in Digital Engineering environments.
   - **5.2.** Study selected methods, not all possible engineering tools.
   - **5.3.** Use a compact engineering case study.
   - **5.4.** Do not build a full industrial Digital Engineering platform.
   - **5.5.** Do not replace domain experts or engineering judgement.
   - **5.6.** Use FMEA only as the demonstration task.
6. **Contributions**
   - **6.1.** Define model integrity in practical engineering terms.
   - **6.2.** Review methods that support model integrity.

6.3. Propose a framework for improving model integrity.

6.4. Demonstrate the framework through an FMEA case study.

6.5. Position AI as one supporting method within the broader framework.

7. Thesis Structure

7.1. Chapter 2 reviews the literature.

7.2. Chapter 3 presents the methodology and framework.

7.3. Chapter 4 applies and evaluates the framework.

7.4. Chapter 5 concludes the thesis.

Chapter 2

Literature Review

This chapter reviews the literature that motivates the thesis focus on improving model integrity in Digital Engineering environments. It begins with the broader engineering transition from document-centric practice toward connected digital models and lifecycle information. It then explains Model-Based Systems Engineering (MBSE) and descriptive modelling notations as the principal means by which such information is represented. The review subsequently identifies a practical problem that remains insufficiently resolved in the literature: the existence of models does not by itself guarantee that model information remains reliable, traceable, current, consistent, or usable for downstream engineering analysis. The chapter therefore positions *model integrity* as the central concept. It then reviews semantic representations, ontologies, graph-based approaches, and artificial intelligence as complementary supporting methods that can help address aspects of this problem. Finally, it introduces Failure Mode and Effects Analysis (FMEA) as the demonstration task used in this thesis to evaluate the consequences of weak or improved model integrity.

2.1 History of Digital Engineering

Systems engineering has historically been conducted through specifications, reports, interface control documents, spreadsheets, review packs, test records, and other document artefacts distributed across teams and tools. These artefacts remain important because they support communication, contractual exchange, certification, and review. However, document-centric practice makes it difficult to maintain coherence across requirements, architecture, interfaces, assumptions, analyses, and verification evidence when systems become large, multidisciplinary, and iterative [9, 25, 5]. Information can be duplicated in multiple places, relationships may remain implicit, and updates often propagate unevenly across lifecycle artefacts. In such settings, the engineering issue is not only data volume but also the difficulty of maintaining shared meaning, current status, and justified decisions across time.

This problem becomes more significant for contemporary socio-technical systems because engineering work is distributed across disciplines, suppliers, lifecycle stages, and digital tools. A local change in one domain may affect interfaces, constraints, analyses, tests, and supporting evidence elsewhere. As a result, the quality of engineering decisions increasingly depends on whether information can be located, interpreted, connected, and updated across the lifecycle rather than on whether individual documents exist in isolation [35, 37, 31].

Digital Engineering has emerged as a response to this need for lifecycle-connected engineering information. The United States Department of Defense defines Digital Engineering as an integrated digital approach that uses authoritative sources of system data and models as a continuum across disciplines to support lifecycle activities from concept through disposal [35]. DoDI 5000.97 extends this position operationally by describing digital engi-

neering as the use and integration of digital models and underlying data to support system development, test and evaluation, and sustainment, while moving the primary means of communicating system information from documents to digital models and their underlying data [37]. In this view, models are not merely alternative documentation. They are intended to become central carriers of engineering information, linked to the data, artefacts, and processes needed for decision-making across the lifecycle.

Two related concepts are especially important. The first is the *digital thread*. Singh and Willcox describe the digital thread as a data-driven architecture that links information generated from across the product lifecycle and is envisaged as the primary or authoritative data and communication platform for a product at a given point in time [31]. DoDI 5000.97 similarly states that digital threads connect authoritative data and orchestrate digital models and information across a system's lifecycle, providing the capability to access, integrate, and transform data into actionable information [37]. The second concept is the *authoritative source of truth*. DoDI 5000.97 defines this as the reference point for models and data across the system life cycle, providing traceability as the system evolves, capturing historical knowledge, and connecting configuration-controlled versions of models and data [37]. Importantly, this should not be understood as a claim that all information must reside in one repository. Rather, the literature supports a more practical interpretation in which authority depends on trust, governance, traceability, and controlled connectivity across sources [37, 31, 14].

This distinction helps separate *digitising documents* from *creating connected engineering information*. Scanning or storing engineering documents digitally may improve access, but it does not by itself create explicit identifiers, dependency relations, versioned traces, semantic alignment, or machine-actionable connections between requirements, design decisions, analysis artefacts, and evidence. By contrast, Digital Engineering aims to support engineering work with information that is visible, accessible, understandable, linked, trustworthy, interoperable, and secure [36, 37]. The value of this shift lies not in replacing every document, but in making engineering information sufficiently connected and current that it can support reasoning, change propagation, and reviewable decisions over time.

For this thesis, the key implication is that Digital Engineering raises the standard for engineering information. Once downstream analyses rely on connected models and digital threads, poor connectivity, weak traceability, stale assumptions, or broken evidence links are no longer minor documentation inconveniences. They become obstacles to trustworthy engineering analysis and to the maintenance of that analysis after model changes.

Table 2.1: Shift from document-centric engineering to MBSE and Digital Engineering

Paradigm	Primary artefact	Main strength	Main limitation	Relevance to model integrity
Document-centric systems engineering	Specifications, reports, spreadsheets, drawings	Familiarity, contractual readability, review support	Fragmented relationships, duplicated information, manual update burden	Integrity problems remain hidden until downstream work is reconciled manually
MBSE	System models and model relations	Stronger structure, traceability, analysis integration, stakeholder communication	Benefits depend on method, governance, and tool discipline	Improves information structure but does not guarantee that model content stays current or trustworthy
Digital Engineering	Connected models, data, simulations, services, and lifecycle traces	Supports lifecycle continuity, reusable information, and data-driven decisions	Strong dependence on semantics, interoperability, governance, and curation	Makes integrity a first-order engineering concern because downstream work depends directly on connected digital information

2.2 Model-Based Systems Engineering

MBSE is widely treated as the principal systems-engineering foundation for Digital Engineering. Estefan's influential INCOSE survey defined MBSE as the formalised application of modelling to support system requirements, design, analysis, verification, and validation activities beginning in the conceptual design phase and continuing across the development lifecycle and later life phases [9]. Madni and Sievers similarly describe MBSE as a systems-engineering approach centred on the evolving system model and motivated by the need to manage rising system scale and complexity while maintaining consistency and traceability [25].

In practical terms, MBSE aims to represent the system in ways that are rich enough to support engineering reasoning rather than only documentation. Official SysML descriptions and MBSE practice literature show that such models may capture requirements, structure, behaviour, interfaces, properties and parameters, analysis cases, verification cases, and relationships among them [9, 27, 29]. This representational breadth is important because many downstream engineering problems require coordinated access to more than one kind of information. For example, a change to a requirement may have implications for architectural elements, allocated functions, parameters, simulations, test cases, and associated evidence.

The literature commonly associates MBSE with several benefits. First, models can improve communication by giving stakeholders shared abstractions of the system and its decomposition [25, 20]. Second, they can improve traceability by making relations among requirements, design constructs, analyses, and verification artefacts more explicit [9, 25]. Third, they can support consistency management by reducing reliance on disconnected descriptions and by enabling more systematic reuse, transformation, and checking [25, 24]. Fourth, they can support earlier analysis and verification by making behaviour, structure, and constraints available in forms that can be inspected, simulated, or transformed before costly implementation steps [25, 29]. Finally, they can provide a basis for lifecycle support and impact analysis, since a change can, in principle, be followed through model relations rather than rediscovered manually across disconnected documents [20, 24].

These benefits, however, should not be overstated. MBSE literature consistently notes that the existence of models does not ensure successful outcomes. Huldt and Stenius show that MBSE adoption and perceived value vary significantly across organisations and depend on how it is implemented in practice [20]. De Saqui-Sannes et al. argue that MBSE must be understood through the combined choice of *languages*, *tools*, and *methods*, because weaknesses often arise at their intersections rather than from notation alone [5]. Likewise, toolchain reviews emphasise interoperability, lifecycle integration, transformation, and traceability as recurring concerns rather than solved problems [24]. Put differently, MBSE can improve the structure and availability of engineering information, but it does not automatically guarantee that the resulting model information is reliable, current, justified, or easy to maintain after change.

That distinction is central to this thesis. A project may have an MBSE repository and still suffer from stale parameter values, missing trace links, unrecorded assumptions, locally meaningful but globally inconsistent terms, or analysis results that cannot be reconnected to revised model content. In such cases, the engineering problem is not absence of models. It is insufficient integrity of the information within and between those models.

2.3 Descriptive Modelling Notations

Descriptive modelling notations are the means by which engineering information is organised, externalised, shared, and manipulated in MBSE practice. They matter because they shape the granularity of model elements, the kinds of relations that can be made explicit,

the degree of precision that can be achieved, and the extent to which model content can be exchanged or processed by tools. A good notation does not solve the whole engineering problem, but it strongly influences what can be represented clearly and what remains hidden in conventions or free text [5, 25].

SysML v1 became the dominant general-purpose systems modelling language in MBSE because it extended UML with facilities that are recognisably relevant to systems engineering, including requirements, structural composition, interfaces, behaviours, parametrics, and allocation relations [27, 11]. Its broad uptake made it an important descriptive notation for architecture representation and traceability-oriented modelling across sectors. The lasting contribution of SysML v1 is therefore not that it solved all systems-engineering modelling problems, but that it provided a shared descriptive language around which tools, practices, tutorials, and organisational methods could develop.

At the same time, the literature also shows why SysML v1 should not be treated as sufficient for the integrity problem addressed in this thesis. Historically, its semantics were only partially precise, many practical modelling decisions remained method-dependent, and interoperability across tools was limited by uneven implementations and fragmented interfaces [2, 5]. Thus, even when teams used the same notation, meaning and usage patterns could still vary across contexts. A model might be diagrammatically complete enough for communication while remaining weakly structured for automation, transformation, or robust cross-tool integration.

SysML v2 and KerML represent a substantial development. The formal OMG material describes KerML as an application-independent modelling language with a well-grounded formal semantics, and SysML v2 as a systems engineering specialisation built on that foundation [28, 29]. Official OMG material and the supporting literature emphasise several intended improvements over SysML v1: greater precision and expressiveness, improved consistency of the language concepts, complementary textual and graphical representations, and standardised APIs and services for accessing and operating on KerML-based and SysML-based models [2, 29, 30]. These developments are especially relevant in Digital Engineering environments because they improve the conditions for automation, model transformation, repository access, fine-grained integration, and model-based services.

The significance of SysML v2 for this thesis lies in the kind of support it can provide for higher model integrity. A more precise language and a standard API can make it easier to create model elements that are better structured, more consistently interpreted, and more readily available for downstream consumption. They can reduce some ambiguity at the notation and tooling layers, and they can provide a better substrate for traceability, query, and transformation services [2, 30]. However, the literature also cautions against a simplistic "language-upgrade" narrative. Almeida et al. show that the semantic foundation of KerML and SysML v2 is richer and more demanding than a straightforward replacement story suggests [1]. Better formal underpinnings improve the potential for rigor, but they do not by themselves solve semantic alignment across domains, curate evidence, enforce modelling governance, or ensure that downstream analyses remain maintainable after changes in the source model.

The practical distinction, therefore, is between *representing information* and *maintaining trustworthy connected information*. Descriptive notations help engineers represent requirements, structure, behaviour, interfaces, and analytical context in more systematic ways than free-text documents usually allow. Yet model integrity also depends on whether terminology is consistent, whether identifiers and links are maintained, whether evidence chains are preserved, whether version changes are reflected in downstream artefacts, and whether model content can be queried and reused without brittle manual reinterpretation. Descriptive notations are essential for this goal, but they are not sufficient on their own.

Table 2.2: MBSE, SysML, and SysML v2 contributions to model integrity

Approach	What it helps represent	Contribution to model integrity	Remaining limitation
MBSE as engineering approach	Requirements, architecture, behaviour, interfaces, analysis and V&V relations	Provides a model-centred basis for connected engineering information	Does not ensure that model content is current, semantically aligned, or evidence-linked
SysML v1.x	Widely used descriptive representation of requirements, structure, behaviour, and parametrics	Makes relations more explicit than document-centric practice and supports traceability-oriented modelling	Semantic interpretation and interoperability often remain tool- and method-dependent
KerML + SysML v2	More precise foundations, complementary textual/graphical modelling, model services, standard APIs	Improves precision, automation potential, interoperability support, and access to models as data	Still does not by itself guarantee consistency, governance, evidence quality, or maintainability after change

2.4 Model Integrity in Digital Engineering

Although the literature regularly discusses model quality, consistency, traceability, interoperability, and trust in related ways, there is no single universally adopted definition of *model integrity* in Digital Engineering. For the purposes of this thesis, model integrity is defined as *the reliability, traceability, currency, consistency, and usability of model information for downstream engineering analysis*. This is a practical rather than purely theoretical definition. It is intended to capture the properties that make model information useful when engineers need to analyse, justify, revise, and maintain downstream work as system information evolves.

Reliability means that the information can be trusted sufficiently for engineering reasoning. This does not require that every model be perfect or final, but it does require that the status, provenance, and intended use of the information are clear enough that engineers can judge whether it is fit for purpose [37, 36]. *Traceability* means that claims, outputs, and decisions can be linked back to the model elements, assumptions, and supporting evidence on which they depend [25, 37]. *Currency* means that model information remains valid, or at least recognisably out of date, when upstream changes occur. It therefore includes change propagation, version control, and visibility of what needs to be revisited after a modification [37, 36]. *Consistency* means that relations, identifiers, and terminology do not conflict across the connected model environment [36, 6]. Finally, *usability* means that downstream tasks can actually consume the model information. Information may be present somewhere in a repository and still be unusable if it is scattered, weakly linked, semantically unclear, or difficult to retrieve in a form needed for analysis.

This emphasis is necessary because engineering models evolve continuously. Requirements are clarified, interfaces change, parameters are recalibrated, simulations are rerun, test results accumulate, and assumptions are revised. Under these conditions, the central practical question is not only whether a model is syntactically valid, but whether the information relevant to later analysis can still be found, understood, justified, and updated when the source model changes. DoDI 5000.97 is explicit that programmes should maintain digital models and associated data throughout the lifecycle so that stakeholders can extract and analyse consistent and up-to-date system information, and that authoritative sources

of truth should remain current, consistent, and enduring [37]. This policy language closely matches the integrity concerns addressed in the present thesis.

Model integrity failures tend to appear in immediately recognisable engineering forms. Missing links between requirements and architecture make it difficult to judge whether a downstream analysis covers the revised design. Implicit assumptions buried in notes, emails, or analyst memory mean that a later user may not know why a certain parameter value or failure interpretation was chosen. Stale analysis outputs can remain attached to a model even after the source configuration has changed. Inconsistent terminology across teams or tools can make components, functions, operating states, or evidence sets appear different when they are not, or equivalent when they should not be. Broken evidence chains make it difficult to justify why a conclusion, risk statement, or recommendation should still be trusted [3, 37, 7]. In each case, the problem is not simply bad modelling style. It is degradation of the engineering information needed for downstream reasoning.

The downstream consequences are substantial. Analyses become slow because engineers must manually reconstruct context that ought to have been explicit. They become fragile because small upstream changes can silently invalidate large portions of the reasoning chain. They become difficult to justify because provenance is incomplete or disconnected. They also become difficult to maintain because each revision requires rediscovering what earlier work depended on. Weak model integrity therefore creates a time-dependent engineering burden: as the system evolves, earlier analyses become progressively harder to reuse or defend, even when the original technical reasoning was sound. This is one reason why the thesis places model integrity at the centre rather than treating it as a secondary quality property of models.

The practical thesis contribution is not to claim a universal taxonomy, but to provide a working integrity concept tied directly to downstream engineering use. In this sense, model integrity is less about abstract conformance and more about whether model information continues to support engineering tasks after the inevitable changes of real projects.

Table 2.3: Practical model integrity concerns in this thesis

Integrity concern	**Definition**	**Typical failure symptom**	**Effect on downstream analysis**
Reliability	Information can be trusted for its intended analytical use	Unclear provenance, uncertain status, unsupported values or results	Conclusions are hard to defend and may be quietly unreliable
Traceability	Outputs and claims can be linked to source model elements and evidence	Missing satisfy/verify links, unexplained assumptions, broken evidence paths	Analysts must reconstruct rationale manually and coverage becomes doubtful
Currency	Information remains valid after model and evidence changes	Stale analysis tables, outdated parameters, unrevised assumptions	Earlier results become partially invalid or misleading after change
Consistency	Terminology and relationships do not conflict across connected artefacts	Duplicate concepts, contradictory identifiers, incompatible relations	Queries, transformations, and comparisons become brittle or ambiguous
Usability	Information can actually be consumed by downstream tasks	Relevant content exists but is scattered, implicit, or inaccessible	Analysis becomes slow, manual, and difficult to repeat or maintain

2.5 Semantic Representations and Ontologies

Descriptive syntax alone is insufficient when engineering information must remain connected across tools, domains, and lifecycle stages. A model element may be structurally well formed while its intended meaning remains ambiguous outside the local modelling convention in which it was created. This is why a substantial part of the literature turns from notation to explicit semantics.

Foundational ontology literature offers a useful starting point. Gruber's well-known definition describes an ontology as an explicit specification of a conceptualisation [12]. Guarino and colleagues later sharpened this view by treating computational ontologies as formal artefacts used to model relevant entities and relations in a system of interest, while also emphasising shared conceptualisation and machine-readable representation [13]. Noy and McGuinness place the practical engineering value clearly: an ontology defines the data and structure that other programs can use, and enables formal analysis, reuse, and extension of domain knowledge [26]. In engineering terms, ontologies are therefore useful when teams need more than a notation. They need shared, inspectable meanings for the concepts and relations that recur across modelling tasks.

Knowledge graphs and related graph-based representations extend this value into settings where multiple heterogeneous datasets, models, and identifiers must be connected. Hogan et al. describe knowledge graphs as graph-based structures designed to support schema, identity, validation, and querying across diverse data [16]. This matters in Digital Engineering because relevant information is often distributed across system models, requirements tools, simulation assets, test repositories, configuration records, and operational data sources. Graph-based representations can make those connections explicit rather than leaving them embedded in tool-specific formats or naming conventions.

The literature suggests several ways in which semantic representations can support model integrity. First, they can clarify *meaning*. If concepts such as function, component, interface, requirement, evidence item, analysis result, or operational state are explicitly represented, then later users and tools have a better basis for interpreting relations consistently [13, 6]. Second, they can expose *relationships and dependencies*. Rather than storing traceability in isolated pairwise links, graph and ontology approaches can reveal broader dependency structures that are traversable and queryable [14, 7]. Third, they can support *reuse and integration* across heterogeneous sources by providing a layer of semantic alignment above local schemas or tool formats [6, 14]. Fourth, they can support *checking and verification*. Dunbar et al. show that ontology-aligned graph data can be used for higher-level verification using open-world and closed-world views together with graph algorithms, providing a tool-agnostic means of analysing connected engineering data [7]. Fifth, they can support *evidence and digital thread queries*, since graph structures are well suited to traversing linked lifecycle information [14, 6].

These advantages are directly relevant to model integrity. Reliability benefits when model meanings are explicit and inspectable. Traceability benefits when relations are queryable rather than buried in disconnected sources. Currency benefits when dependencies can be traversed to assess change impact. Consistency benefits when shared vocabularies and rules constrain how concepts are related. Usability benefits when downstream tasks can retrieve contextual information without relying on manual repository archaeology.

However, semantic representations are not cost free. Ontology development requires agreement on concepts, boundaries, and governance. It introduces extra modelling effort and ongoing maintenance responsibilities. Domain-specific ontologies may be difficult to generalise; overly ambitious ontological schemes risk becoming hard to apply consistently; and the benefits may not justify the overhead for every project or subsystem [26, 13]. There is also a risk of over-engineering: an ontology that is formally elegant but poorly aligned with the actual engineering workflow may add complexity without improving practice. For this reason, the thesis treats ontologies and graph-based representations as enabling methods

that can support model integrity when the problem requires explicit semantics, integration, and queryable dependency structures.

Table 2.4: Semantic representations and ontologies for model integrity

Method	Contribution	Example use	Limitation
Ontology	Defines concepts, properties, and relations explicitly	Clarifying what counts as a function, component, failure, evidence item, or verification relation	Requires modelling effort, agreement, and governance
Knowledge graph	Connects heterogeneous entities and relations in queryable graph form	Traversing requirement–function–component–analysis–evidence chains	Quality depends on identifiers, mapping quality, and maintenance
Semantic mapping	Aligns local tool schemas and vocabularies to shared meaning	Relating model elements from different repositories or disciplines	Mapping work can be brittle and domain-specific
Rule/query layer	Adds validation, reasoning, and retrieval over connected representations	Detecting missing links, inconsistent patterns, or change impact candidates	Rules need maintenance and can be incomplete with respect to tacit engineering judgement

2.6 Artificial Intelligence as a Supporting Method

Artificial intelligence is relevant to this thesis, but it is not the research goal. The literature suggests that AI, and especially large language models (LLMs), can be useful for engineering tasks that are language-heavy, search-heavy, or dependent on heterogeneous context, but that such use must be handled with caution in engineering settings where correctness, provenance, and accountability matter [4, 10].

Current literature indicates several practical tasks for which LLMs can offer support. These include summarisation of large artefact sets, classification of requirements or issues, extraction of structured information from text, generation of candidate links or candidate explanations, review support, and the drafting of intermediate artefacts that still require expert validation [4, 8]. In requirements-related studies, for example, LLMs have been shown to assist in reformulating textual requirements and assessing their adherence to quality criteria, with promising but not definitive results [8]. Such tasks are relevant to Digital Engineering because much engineering context still arrives in mixed forms: structured models, semi-structured tables, descriptive text, rationale, and legacy documents.

The literature is equally clear that risks remain substantial. Hallucination surveys describe a persistent tendency for LLMs to generate plausible but nonfactual or unsupported content [17]. Ferrari and Spoletini argue that concerns around correctness, fairness, and trustworthiness remain central when LLMs are used for requirements-engineering tasks [10]. In Digital Engineering contexts, these concerns translate into familiar integrity problems: unsupported claims weaken reliability; missing provenance weakens traceability; overconfident explanations obscure uncertainty; and partial context can cause the model to produce answers that appear coherent while silently omitting critical dependencies.

For these reasons, the most credible role for AI in this thesis is *bounded support*. AI can help with extraction, classification, summarisation, candidate generation, and explanation support, but it should do so within a structure supplied by the engineering environment

rather than as an unconstrained generator of engineering truth. Buchmann et al. explicitly frame LLM relevance in relation to semantics-driven systems engineering, that is, in relation to engineered knowledge structures rather than free-form generation alone [4]. This aligns closely with the framework logic pursued by the thesis.

Three forms of bounding are especially important. First, AI should be grounded in *structured model context*: model elements, identifiers, relationships, and parser outputs should constrain what the model sees and what it is asked to do. Second, AI should be informed by *semantic guidance*: ontologies, vocabularies, and graph context can reduce ambiguity and help place candidate outputs in an explicit meaning structure [4]. Third, AI should remain tied to *retrieved evidence and reviewable provenance*. Retrieval-augmented generation is relevant here because it supplements model parameters with external retrieved information, thereby providing a better basis for source-aware responses than parameter-only generation [23]. Even then, human review remains necessary, because retrieval does not remove the need for interpretation or guarantee that the generated synthesis is correct.

The appropriate conclusion for this thesis is therefore modest but useful. AI can support selected activities that contribute to model integrity, especially where engineering information must be processed across structured and unstructured sources. It may improve speed and assist discovery. It may help suggest candidate links or candidate failure information. But it does not replace engineering judgement, and it should not be treated as an authoritative source of truth. In this thesis it is positioned as one enabling method that becomes more useful when combined with descriptive models, semantic representations, evidence links, and explicit review.

Table 2.5: Role of AI as a supporting method

AI-supported task	Possible contribution	Risk	Required control
Extraction	Pulling candidate entities, properties, or relations from text and mixed artefacts	Omission or fabricated extraction	Structured input context and human verification
Classification	Sorting requirements, issues, evidence, or change items into useful categories	Misclassification caused by limited context	Clear schema, examples, and traceable outputs
Summarisation	Condensing large artefact sets for review and orientation	Loss of nuance or unsupported compression	Source retrieval and review against originals
Candidate generation	Suggesting links, failure modes, causes, effects, or rationale text	Hallucinated content presented as fact	Evidence binding, constrained prompts, expert approval
Explanation	Producing accessible descriptions of model content or reasoning paths	Overconfident explanations that exceed evidence	Citation to retrieved context and uncertainty-aware use
Review and revision support	Flagging incompleteness, ambiguity, or update candidates	False confidence and missed defects	Human-in-the-loop judgement and explicit acceptance criteria

2.7 Failure Mode and Effects Analysis

Failure Mode and Effects Analysis is introduced in this thesis as a *demonstration task*, not as the thesis goal. IEC 60812:2018 describes FMEA as a systematic method for establishing how items or processes might fail to perform their function so that treatments can be identified, and explains how FMEA and FMECA are planned, performed, documented, and maintained [21]. MIL-STD-1629A defines FMEA as a procedure by which each potential failure mode in a system is analysed to determine its results or effects on the system and to classify the potential failure mode according to severity [34]. Across these traditions, FMEA is best understood as a structured engineering analysis that relates system items and functions to possible failure modes, causes, effects, controls, and prioritisation information.

A credible FMEA depends on connected engineering information. At minimum it requires a sufficiently clear understanding of the item or function under study, the context in which it operates, plausible failure modes, local and higher-level effects, and the reasoning or evidence supporting those judgements [21, 34]. In many practical forms it also includes occurrence and detection-related assessments, controls, recommended actions, and supporting rationale. These inputs are rarely located in one place. They are usually distributed across requirements, architecture descriptions, interface information, behaviour models, analyst assumptions, verification artefacts, previous incidents, and domain knowledge. This makes FMEA a particularly informative case when the quality of connected engineering information is under examination.

The literature repeatedly characterises FMEA as labour-intensive, expert-dependent, and often still document-centric in practice. Spreafico et al. identify a long history of attempts to improve and adapt FMEA, indicating that important challenges remain despite the maturity of the method [32]. Recent reviews argue that conventional FMEA becomes increasingly inadequate as systems grow in complexity because the method remains manual, document-heavy, and difficult to keep aligned with evolving system knowledge [39]. This is one reason why researchers have explored MBSE-assisted FMEA, ontology-based support, and more recently AI-assisted support [19, 18, 22, 38].

These research directions are useful for the present thesis because they show how strongly FMEA depends on model integrity. If functions are missing or weakly defined, failure modes may be omitted. If component–function relations are unclear, causes and effects become speculative or duplicated. If terminology is inconsistent, similar failure modes may be entered multiple times under different names. If assumptions are stale or analysis results are no longer linked to the current model configuration, the FMEA becomes difficult to justify. If evidence chains are weak, ratings and recommended actions become harder to defend. If a model changes and dependencies are not explicit, updating the FMEA becomes a manual and fragile exercise [18, 15, 38]. In short, FMEA quickly reveals whether connected engineering information is usable enough to sustain a downstream analysis.

This is why FMEA is suitable as the chapter's demonstration task. First, it produces a structured and recognisable output, which makes success and failure visible. Second, it is engineering-relevant rather than an artificial benchmark. Third, it is auditable: unsupported causes, effects, and classifications are comparatively easy to spot. Fourth, it is strongly dependent on connected information spanning functions, structures, interfaces, assumptions, and evidence. Fifth, it is sensitive to change: even modest design updates can require significant rework when the underlying information web is weakly maintained. For these reasons, FMEA is appropriate as an evaluation surface for a model integrity framework, even though the thesis is not primarily about FMEA automation.

Table 2.6: Why FMEA is suitable as a demonstration task

FMEA characteristic	Why it depends on model integrity	What failure reveals
Structured output	FMEA rows depend on explicit items, functions, causes, effects, and rationale	Missing or weakly linked model content becomes visible quickly
Engineering relevance	FMEA must reflect actual system meaning rather than generic text patterns	Superficial automation produces plausible but weakly justified content
Auditability	Entries can be checked against source architecture, assumptions, and evidence	Unsupported claims, duplications, and omissions are easier to detect
Sensitivity to connected information	Good FMEA requires linked requirements, functions, interfaces, controls, and evidence	Weak traceability and unclear semantics degrade quality directly
Sensitivity to change	Model changes should trigger reconsideration of dependent failure knowledge	Stale tables and manual rework expose weak currency and maintainability

2.8 Research Gap

The literature reviewed in this chapter supports a clear overall argument. Digital Engineering aims to move engineering practice beyond disconnected documents toward lifecycle-connected digital models and data that can support decision-making across development, verification, operation, and sustainment [35, 37, 31]. MBSE provides the main engineering foundation for this shift by representing requirements, structure, behaviour, interfaces, parameters, and verification relations in model form [9, 25]. Descriptive notations, especially SysML, give this information a common representational structure, and recent developments in SysML v2, KerML, and standard APIs improve precision and interoperability potential [27, 29, 28, 30]. However, the literature also makes clear that models and notations alone do not guarantee that engineering information remains reliable, traceable, current, consistent, or usable after changes [20, 5, 24].

Semantic representations and ontologies address part of this gap by making concepts, relations, and dependencies explicit in machine-interpretable form, while graph-based representations make heterogeneous engineering information more queryable and traversable [12, 13, 16, 6, 7]. Artificial intelligence can support selected information-processing tasks, especially where engineering context is partly textual or heterogeneous, but current literature strongly cautions that trustworthiness depends on grounding, structure, and review rather than on unconstrained generation [4, 10, 17, 23]. FMEA, meanwhile, is a useful demonstration surface because it is strongly information-dependent and makes the practical consequences of weak model integrity immediately visible [21, 34, 32].

The research gap follows from the way these areas are typically discussed. The literature contains many useful methods, but often in separation: Digital Engineering vision and policy; MBSE methods; modelling languages; ontology-aligned integration; graph-based verification; AI support for selected tasks; and FMEA automation or augmentation. What is much less developed is a practical framework that organises these methods around the problem of *improving model integrity* in Digital Engineering environments and preserving the usefulness of downstream engineering analysis after source models change. This gap is especially important for real engineering work, where the challenge is rarely the absence of modelling notation or isolated automation, but the difficulty of keeping connected engineering information reliable, traceable, current, consistent, and usable over time.

The next chapter addresses this gap by developing a framework for improving model

integrity in Digital Engineering environments. The framework combines descriptive modelling, semantic representation, evidence-oriented traceability, and bounded AI support. It is then demonstrated and evaluated through an FMEA case study, not because FMEA is the thesis goal, but because it provides a practical and audit-friendly way of observing how model integrity affects downstream engineering analysis.

Chapter 2 Draft Outline

This section are the sentences that later turned into paragraphs in this chapter.

2.1. Digital Engineering and the Shift from Documents to Models

- **2.1.1.** Traditional systems engineering has often relied on document-centric practices.
- **2.1.2.** Complex engineering systems require connected lifecycle information.
- **2.1.3.** Digital Engineering aims to connect models, data, analyses, and evidence.
- **2.1.4.** Concepts such as digital thread and authoritative source of truth support this shift.
- **2.1.5.** The main challenge is not only digitising information, but keeping it connected and usable.

2.2. Model-Based Systems Engineering

- **2.2.1.** MBSE provides a foundation for Digital Engineering.
- **2.2.2.** MBSE represents requirements, functions, structures, behaviours, interfaces, and verification relationships through models.
- **2.2.3.** MBSE can improve communication, traceability, consistency, and lifecycle support.
- **2.2.4.** However, MBSE does not automatically guarantee reliable or maintainable model information.
- **2.2.5.** The benefits of MBSE depend on modelling discipline, method, governance, and tool integration.

2.3. Descriptive Modelling Notations

- **2.3.1.** Modelling notations provide structured ways to describe engineering information.
- **2.3.2.** SysML v1 has been widely used for systems modelling.
- **2.3.3.** SysML v2 and KerML aim to improve precision, semantics, APIs, and interoperability.
- **2.3.4.** Descriptive modelling notations help structure information but do not solve all integrity problems.
- **2.3.5.** Semantic consistency, evidence linking, and maintainability still require additional methods.

2.4. Model Integrity in Digital Engineering

- **2.4.1.** Model integrity concerns the reliability, traceability, currency, consistency, and usability of model information.
- **2.4.2.** Model integrity becomes important when models, assumptions, parameters, analyses, and evidence evolve over time.
- **2.4.3.** Common integrity problems include missing links, implicit assumptions, stale outputs, inconsistent terminology, and broken evidence chains.
- **2.4.4.** Poor model integrity makes downstream engineering analysis slower and harder to justify.
- **2.4.5.** A practical understanding of model integrity is needed to support reliable Digital Engineering workflows.

2.5. Semantic Representations and Ontologies

- **2.5.1.** Modelling syntax alone is not enough; engineering meaning must also be explicit.
- **2.5.2.** Ontologies represent concepts, relationships, and constraints in a structured way.
- **2.5.3.** Graph-based representations can make dependencies and evidence links more queryable.
- **2.5.4.** Semantic methods can support model integrity by improving meaning, reuse, and traceability.
- **2.5.5.** Limitations include modelling effort, governance, maintenance, and domain specificity.

2.6. Artificial Intelligence as a Supporting Method

- **2.6.1.** Artificial intelligence can support engineering information processing.

2.6.2. Large language models can assist with summarisation, classification, extraction, explanation, and candidate generation.

2.6.3. AI outputs can be risky when they are unsupported, untraceable, or based on incomplete context.

2.6.4. Hallucination and overconfident reasoning are critical concerns in engineering contexts.

2.6.5. AI should be treated as a supporting method bounded by structured models, semantic context, evidence, and human review.

2.7. Failure Mode and Effects Analysis as Demonstration Task

2.7.1. Failure Mode and Effects Analysis (FMEA) is a structured method for analysing possible failures, causes, effects, controls, and actions.

2.7.2. FMEA requires connected information about functions, components, assumptions, parameters, and evidence.

2.7.3. FMEA is often time-consuming and knowledge-intensive.

2.7.4. Weak model integrity can lead to missing failure modes, duplicated reasoning, unsupported effects, and difficult updates.

2.7.5. FMEA is therefore suitable as a case study for demonstrating how model integrity affects downstream analysis.

2.8. Synthesis and Research Gap

2.8.1. Digital Engineering requires connected and maintainable engineering information.

2.8.2. MBSE and descriptive modelling notations help represent this information.

2.8.3. Semantic representations and ontologies help clarify meaning and relationships.

2.8.4. Artificial intelligence can support selected tasks but must be grounded and controlled.

2.8.5. Existing approaches are often discussed separately rather than as part of an integrated model integrity framework.

2.8.6. The research gap is the need for a practical framework that organises these methods to support higher model integrity.

2.8.7. This thesis addresses the gap by proposing and evaluating such a framework through an FMEA case study.

Chapter 3

Methodology

3.1 Research Design

This thesis adopts a design-oriented research methodology. The objective is to develop a framework for improving model integrity in Digital Engineering environments and to demonstrate its application through a downstream engineering analysis task. The focus is not on replacing engineering judgement, but on examining how model-based information can be structured, bounded, semantically guided, and used to support traceable and reviewable analysis.

The methodology is based on four main elements: model representation, model parsing, ontology-based reasoning guidance, and AI-supported reasoning. The model representation provides the system-specific engineering information. The model parser extracts and structures relevant information from this model. The ontology provides a domain-agnostic reasoning structure for failure analysis. The AI component then uses both the parsed model context and the ontology to support the generation, review, scoring, explanation, and revision of analysis outputs.

The methodology is demonstrated using a toy flashlight system model represented in a textual SysML-style notation. The flashlight model includes requirements, actions, parts, ports, attributes, connections, state behaviour, and requirement allocations. This makes it suitable as a compact case study: it is small enough to remain inspectable, but rich enough to contain the kinds of relationships that are important for model integrity. The full flashlight model is provided in Appendix A.

Failure Mode and Effects Analysis (FMEA) is used as the downstream engineering analysis task. FMEA is selected because it depends strongly on connected engineering information, including system items, functions, failure modes, causes, effects, controls, scores, and supporting evidence. In this thesis, FMEA is not treated as the main research goal. Instead, it is used as a demonstration task for examining how the proposed model integrity framework supports downstream analysis.

3.2 Framework Overview

The proposed framework combines four main elements: model representation, model parser, ontology, and AI-supported reasoning. These elements are not arranged as a simple linear sequence in which the ontology comes after the parser. Instead, the parser and ontology provide two complementary inputs to the AI component.

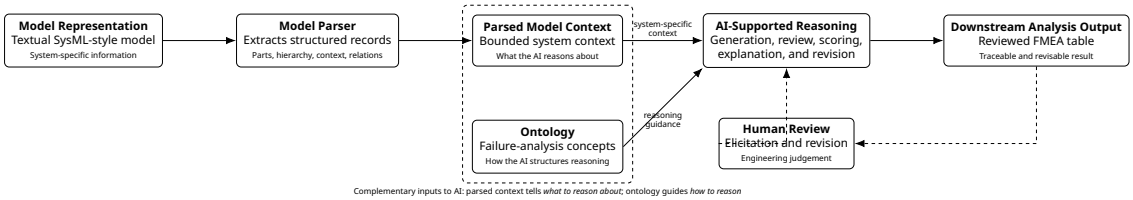


Figure 3.1: Framework overview showing the relationship between model representation, model parsing, ontology-based reasoning guidance, AI-supported reasoning, and downstream analysis output.

The parser provides system-specific context. It tells the AI what system information should be considered. The ontology provides reasoning guidance. It tells the AI how failure-analysis concepts should be interpreted and related. This distinction is important because the AI should not reason from the complete SysML model as one large unstructured text block, nor should it invent its own reasoning structure.

The framework can therefore be understood as follows:

- the model representation provides the system-specific engineering information;
- the model parser extracts bounded and relevant model context;
- the ontology provides domain-agnostic reasoning structure;
- the AI component combines the parser output and ontology guidance to support downstream analysis.

Table 3.1 summarises the role of each framework component.

Table 3.1: Main components of the proposed model integrity framework

Component	Role in the framework	Contribution to model integrity
Model representation	Provides the system-specific engineering information.	Captures system structure, behaviour, requirements, functions, interfaces, and allocations.
Model parser	Extracts and structures relevant context from the model.	Makes model information bounded, explicit, and usable for downstream reasoning.
Ontology	Provides domain-agnostic reasoning concepts and relationships.	Guides how failure-analysis concepts should be understood and related.
AI-supported reasoning	Uses parsed model context and ontology guidance to support analysis.	Assists with generation, review, explanation, scoring, and revision while remaining grounded in structured inputs.
Human review	Allows the user to inspect, question, and revise outputs.	Preserves engineering accountability and supports correction of incomplete or weak results.

3.3 Model Representation

The first element of the framework is the model representation. In this thesis, the system is represented using a textual SysML-style model. The model describes the toy flashlight system and includes several types of engineering information. These include requirements,

actions, parts, attributes, ports, connections, bindings, state transitions, and requirement allocations.

The flashlight model contains three main packages. The first package describes the requirements of the flashlight, such as field of view, light power, size, and weight. The second package describes the action tree for producing directed light, including actions such as providing DC power, connecting DC power, generating light, and directing light. The third package describes the parts tree, including the flashlight, batteries, switch, lamp, optics, reflector, lens, and structure. A further package allocates requirements to design features.

In the framework, the model is not treated only as documentation. It is treated as the source from which structured analysis context is derived. This is important because downstream analysis depends on more than the existence of a model. It depends on whether relevant model information can be found, interpreted, connected, and reused. For example, an FMEA row about the lamp should ideally be supported by information about the lamp itself, its ports, its performed action, its connection to the switch and optics, and any relevant requirements or allocations.

The model representation therefore provides the starting point for model integrity support. However, the raw textual model is not directly suitable as AI input because it contains many nested structures, comments, declarations, and relationships. A parser is therefore introduced to transform the raw model into structured model records.

3.4 Model Parser

The second element of the framework is the model parser. The parser transforms the raw textual SysML-style model into structured part-level records. Each record corresponds mainly to a SysML part and contains the part name, parent part, hierarchical layer, original model text, and related context.

The parser is necessary because the AI component should not process the full model as one large unstructured text block. Large unstructured inputs can hide relevant relationships, increase noise, and make the reasoning process harder to inspect. The parser therefore provides bounded system-specific context. In simple terms, the parser defines what the AI should think about.

The parser is designed to handle both normally formatted SysML-style text and collapsed one-line model text. This is useful because model text may be copied from different tools, exported in different formats, or processed through automation workflows. The parser therefore uses comment masking, brace matching, declaration parsing, parent-child assignment, layer computation, and context extraction.

The output schema of the parser is shown in Table 3.2.

Table 3.2: Output schema of the SysML model parser

Field	Description
`part_name`	Name of the SysML part.
`parent_part`	Name of the nearest parent part, if one exists.
`layer`	Hierarchical layer of the part, where leaf parts have layer 0.
`part_chunk`	Original SysML text belonging to the part declaration.
`context_text`	Related actions, requirements, ports, attributes, connections, bindings, flows, allocations, and state/control statements.

The parser does not only extract part declarations. It also attempts to recover the engineering context around each part. For example, if a part performs an action, the corresponding action body is included in the part context. If a part is mentioned in a connection, binding, flow, or requirement allocation, that relation is added to the context. This creates

a compact representation that is more useful for downstream analysis than the raw model alone.

Algorithm 1 presents the model parser in pseudocode form.

Algorithm 1 SysML Model Parser

Require: Raw SysML-style textual model M
Ensure: Parsed part records $\mathscr{P}$

$M \leftarrow$ strip code fences, normalise line endings, and remove surrounding whitespace
$M_{masked} \leftarrow$ mask line comments and block comments while preserving character positions
$\mathscr{B}_{part} \leftarrow$ parse all `part` and `ref part` declarations in M_{masked}
$\mathscr{B}_{action} \leftarrow$ parse all `action` declarations in M_{masked}
$\mathscr{B}_{req} \leftarrow$ parse all `requirement` declarations in M_{masked}
for all declaration b in $\mathscr{B}_{part} \cup \mathscr{B}_{action} \cup \mathscr{B}_{req}$ **do**
    if b has an opening brace **then**
        Find the matching closing brace using brace-depth counting
        Store declaration name, start index, end index, header, body, and full text
    else
        Store declaration as a single-line declaration
    end if
end for
for all part $p \in \mathscr{B}_{part}$ **do**
    Assign the nearest containing part as `parent_part`
    Assign all directly contained parts as children of p
end for
for all part $p \in \mathscr{B}_{part}$ **do**
    if p has no child parts **then**
        Set $\texttt{layer}(p) \leftarrow 0$
    else
        Set $\texttt{layer}(p) \leftarrow 1 + \max(\texttt{layer}(c))$ for all children c of p
    end if
end for
$\mathscr{S} \leftarrow$ extract global relation statements from M_{masked} ▷ `connect`, `bind`, `flow`, `perform`, `allocate`, `transition`
$\mathscr{P} \leftarrow \emptyset$
for all part $p \in \mathscr{B}_{part}$ **do**
    $D_p \leftarrow$ body of p with nested child-part blocks removed
    $F_p \leftarrow$ extract local features from D_p ▷ Ports, attributes, performed actions, bindings, connections, and states
    $N_p \leftarrow$ build related-name set for p ▷ Part name, path, ports, attributes, actions, and child names
    $\mathscr{A}_p \leftarrow$ select actions from $\mathscr{B}_{action}$ that match performed actions in F_p
    $\mathscr{S}_p \leftarrow \emptyset$
    for all statement $s \in \mathscr{S}$ **do**
        if s is not inside an unrelated part and s mentions an element in N_p **then**
            Add s to $\mathscr{S}_p$
        end if
    end for
    $\mathscr{R}_p \leftarrow$ select requirements from $\mathscr{B}_{req}$ referenced by allocations or related statements in $\mathscr{S}_p$
    $C_p \leftarrow$ concatenate related actions, local features, relation statements, allocations, requirements, and state/control statements
    Add record with fields `part_name`, `parent_part`, `layer`, `part_chunk`, and `context_text` to $\mathscr{P}$
end for
return $\mathscr{P}$

3.5 Ontology

The third element of the framework is the ontology. The ontology is used as an independent semantic input to the AI-supported reasoning process. It does not come after the model parser in a linear pipeline. Instead, it works in parallel with the parser output. The parser provides system-specific context from the SysML model, while the ontology provides domain-agnostic reasoning concepts and relationships. Therefore, the parser defines what system information the AI should reason about, while the ontology guides how the AI should structure that reasoning.

For the demonstration task, a domain-agnostic Failure Mode and Effects Analysis (FMEA) ontology is used. The ontology was developed from established FMEA and risk-analysis guidance, translated into a human-readable concept outline, formalised into a graph-like JSON structure, and then prepared for GraphRAG-based use. The full ontology structure and creation process are described in Appendix C.

The ontology contains concepts needed for failure-analysis reasoning, including item, function, failure mode, cause, effect, control, detection mechanism, severity, occurrence, detection, recommended action, residual risk, and evidence. It also includes broader reasoning support, such as FMEA process steps, failure domains, failure-mode categories, cause categories, control methods, and AI question templates. These concepts help the AI organise failure analysis in a more systematic way rather than generating rows from unstructured text alone.

The ontology does not replace the SysML model. The SysML model describes the specific system being analysed, such as the flashlight parts, ports, actions, connections, requirements, and allocations. The ontology describes the general logic of FMEA reasoning. This separation is important for model integrity because it keeps system-specific information and domain-general reasoning guidance distinct but connected.

In the framework, the ontology is used for three main purposes. First, it guides the generation of candidate failure modes by providing the expected FMEA concepts and relationships. Second, it supports review by helping check whether generated rows contain coherent item–function–failure mode–cause–effect–control relationships. Third, it supports explanation and revision by giving the chat interface a shared conceptual structure for discussing and improving the FMEA table.

By providing this semantic reasoning layer, the ontology helps make AI-supported analysis more bounded, consistent, explainable, and reviewable. The detailed ontology content, including its standards foundation, reasoning rules, node groups, edge relations, and GraphRAG-ready structure, is provided in Appendix C.

3.6 AI-Supported Reasoning

The fourth element of the framework is AI-supported reasoning. The AI component receives two complementary inputs. The first input is the parsed model context, which provides information about the specific system. The second input is the ontology, which provides the conceptual structure for reasoning.

The parsed model context grounds the AI in the system being analysed. For example, in the flashlight model, the AI can receive context about the battery, switch, lamp, optics, reflector, lens, or structure, including their parent-child relationships, ports, actions, and connections. The ontology then guides the AI in organising this information into failure-analysis concepts such as failure mode, cause, effect, control, and evidence.

The AI is used to support selected reasoning tasks. These include candidate failure mode generation, group-level review, global review, terminology refinement, cause-effect checking, severity-occurrence-detection scoring support, explanation, and revision. However, the AI output is not treated as automatically correct. Human review remains necessary because

downstream engineering analysis requires judgement, interpretation, and accountability.

This separation between parser output and ontology guidance is central to the methodology. It reduces the risk of asking the AI to reason from an unstructured model text and reduces the risk of unsupported reasoning. The AI is therefore used as an assisted reasoning layer built on top of structured model information and semantic guidance.

3.7 Demonstration Through Failure Analysis

The framework is demonstrated through automated failure analysis. Failure Mode and Effects Analysis is selected as the case study because it depends strongly on connected engineering information. FMEA requires information about system items, functions, possible failure modes, causes, effects, controls, severity, occurrence, detection, recommended actions, and supporting evidence.

The toy flashlight model is suitable for this demonstration because it includes both structural and behavioural information. For example, the model describes batteries that provide DC power, a switch that connects power, a lamp that generates light, and optics that direct light. It also includes connections between these parts and requirements related to illumination and physical properties. These features allow the demonstration to examine whether the framework can preserve useful links between system model information and generated analysis outputs.

Weak model integrity can make FMEA incomplete, inconsistent, difficult to justify, or difficult to update. For instance, if a connection between the lamp and optics is not captured, then a failure mode related to loss of light transfer may be missed. If a requirement allocation is not preserved, then the effect of a failure on a requirement may be difficult to justify. FMEA therefore provides a useful evaluation surface for the model integrity framework.

3.8 Failure Analysis Generation Procedure

The failure analysis generation procedure uses the parsed model records and the ontology as complementary inputs to the AI component. The parsed model records define the bounded system-specific context. The ontology defines the reasoning structure. Together, these inputs are used to generate, review, score, and revise a failure-analysis table.

The parsed records are first grouped according to their parent part. This grouping is useful because failures are often meaningful at subsystem or functional-group level rather than only at isolated component level. For example, the child parts of the flashlight, such as the battery, switch, lamp, and optics, are meaningful both as individual components and as part of the larger flashlight function of producing directed light.

For each parent group, the AI receives the grouped model context and the ontology. It then generates candidate FMEA rows for that group. These rows are reviewed at group level to improve relevance, specificity, terminology, and cause-effect consistency. After all groups are processed, the rows are combined into one table. A global review is then performed across all groups to remove duplicates, normalise terminology, and improve consistency. Finally, severity, occurrence, and detection scores are assigned using a scoring rubric.

Algorithm 2 presents the FMEA generation procedure.

Algorithm 2 Ontology-Guided FMEA Generation

Require: Parsed model records $\mathscr{P}$, original model M, FMEA ontology $\mathscr{O}$, AI model A, scoring rubric $\mathscr{R}$
Ensure: Reviewed and scored FMEA table $\mathscr{F}$

$\mathscr{G} \leftarrow$ group records in $\mathscr{P}$ by `parent_part`
$\mathscr{F}_{all} \leftarrow \emptyset$
for all group $g \in \mathscr{G}$ **do**
    $parent_g \leftarrow$ parent part of group g
    $children_g \leftarrow$ all parsed records belonging to $parent_g$
    $X_g \leftarrow$ construct bounded group context from:
     parent name,
     child part names,
     child part chunks,
     child context texts,
     relevant actions, ports, connections, requirements, and allocations
    $\mathscr{F}_g^{raw} \leftarrow A(X_g, \mathscr{O})$ ▷ Generate candidate failure-analysis rows
    $\mathscr{F}_g^{reviewed} \leftarrow A(\mathscr{F}_g^{raw}, X_g, \mathscr{O})$ ▷ Review rows within the same parent group
    Remove vague, duplicated, or unsupported rows inside $\mathscr{F}_g^{reviewed}$
    Improve consistency between item, function, failure mode, cause, effect, control, and evidence
    Add $\mathscr{F}_g^{reviewed}$ to $\mathscr{F}_{all}$
end for
$\mathscr{F} \leftarrow$ combine all reviewed group-level rows from $\mathscr{F}_{all}$
$\mathscr{F} \leftarrow A(\mathscr{F}, M, \mathscr{O})$ ▷ Global review across all groups
Remove duplicated failure modes across groups
Normalise repeated terminology for items, functions, causes, effects, and controls
Check consistency of similar failure modes across the full table
Strengthen weak evidence or rationale where possible
for all row $r \in \mathscr{F}$ **do**
    $r.S \leftarrow$ assign severity score using r, M, $\mathscr{O}$, and $\mathscr{R}$
    $r.O \leftarrow$ assign occurrence score using r, M, $\mathscr{O}$, and $\mathscr{R}$
    $r.D \leftarrow$ assign detection score using r, M, $\mathscr{O}$, and $\mathscr{R}$
    $r.RPN \leftarrow r.S \times r.O \times r.D$
end for
return $\mathscr{F}$

3.9 FMEA Table Structure

The generated FMEA table contains the information required for engineering review. Each row represents one candidate failure mode linked to the parsed model context and guided by the ontology. The table schema is shown in Table 3.3.

Table 3.3: Generated FMEA table schema

Column	Description
Item or part	The model element or component being analysed.
Parent group	The parent part or subsystem used for grouped analysis.
Function	The intended function inferred from the model and context.
Failure mode	The way in which the item or function may fail.
Cause	The possible reason why the failure mode may occur.
Effect	The local or system-level consequence of the failure mode.
Existing control	Existing mechanism that may prevent, detect, or reduce the failure.
Severity (S)	Score representing the seriousness of the effect.
Occurrence (O)	Score representing the likelihood of the cause or failure mode.
Detection (D)	Score representing how difficult the failure is to detect.
RPN	Risk Priority Number calculated from S, O, and D.
Recommended action	Suggested action to reduce risk or improve understanding.
Evidence or rationale	Model-based explanation supporting the generated row.

The Risk Priority Number is calculated as:

$$RPN = S \times O \times D \tag{3.1}$$

where S is severity, O is occurrence, and D is detection.

3.10 Interactive Review and Revision

The generated FMEA table is not treated as a final static result. Instead, it is presented to the user through an interface that supports review and revision. This is important because engineering analysis requires human judgement. The purpose of the interface is to allow the user to understand, challenge, correct, and improve the generated table.

The interaction has two modes: elicitation and revision. In elicitation mode, the user asks questions about the current table. For example, the user may ask why a failure mode was generated, which model element supports a row, why a severity score was assigned, or which failure modes belong to a subsystem. In this mode, the system provides an explanation but does not modify the table.

In revision mode, the user asks the system to modify the FMEA table. Example requests include removing a duplicated failure mode, adding a missing interface failure, rewriting a cause more clearly, changing a score, or splitting one failure mode into two separate rows. In this mode, the system returns both a response to the user and an updated FMEA table.

Table 3.4 summarises the two interaction modes.

Table 3.4: Interactive review modes

Mode	Purpose	Output
Elicitation	Help the user understand the current FMEA table.	Chat response only.
Revision	Modify, correct, or improve the FMEA table.	Chat response and updated FMEA table.

This interaction supports model integrity by keeping analysis outputs reviewable and maintainable. It also ensures that AI-supported outputs remain subject to human judgement rather than being accepted as final automatically.

3.11 Evaluation Method

The framework is evaluated in two complementary ways: verification and validation. In this thesis, verification means checking whether the generated FMEA output satisfies predefined structural and consistency tests. Validation means assessing whether the output appears useful, traceable, maintainable, and understandable for the intended engineering analysis purpose. In simple terms, verification asks whether the generated table is built correctly according to expected rules, while validation asks whether the result is meaningful and useful for engineering review.

3.11.1 Verification Method

The verification step evaluates the generated FMEA table against a set of predefined tests. These tests are written before analysing the output and are used to check whether the generated FMEA satisfies basic structural, model-coverage, and reasoning requirements. Each test is evaluated as passed, partially passed, or failed.

The purpose of verification is not to prove that every generated failure mode is correct. Rather, it checks whether the output follows the expected structure and whether it remains sufficiently connected to the model and ontology. This is important because an FMEA table may look complete while still missing important parts, mixing causes with effects, or failing to provide evidence for generated rows.

Table 3.5 lists the verification tests used in this thesis.

Table 3.5: Verification tests for the generated FMEA table

Test	Verification question	Purpose
T1	Are all relevant model parts represented in the FMEA table?	Checks whether the analysis covers the system elements extracted by the parser.
T2	Are parent-child relationships reflected correctly?	Checks whether failure modes are assigned to the correct component, subsystem, or parent group.
T3	Are functions linked to the correct items or parts?	Checks whether failure modes are derived from what each part is intended to do.
T4	Does each FMEA row contain a clear failure mode?	Checks whether the row states a specific way in which a function or item can fail.
T5	Are failure modes separated from causes and effects?	Checks whether the FMEA avoids mixing what fails, why it fails, and what happens afterward.
T6	Does each row include a plausible cause-effect chain?	Checks whether the stated cause can reasonably lead to the failure mode and whether the effect follows from it.
T7	Are interface-related failures considered where connections exist?	Checks whether connections, ports, flows, and bindings are used to identify possible interface failures.
T8	Are relevant requirements or constraints reflected in the analysis?	Checks whether requirement allocations, constraints, or performance limits influence the generated FMEA where applicable.
T9	Are severity, occurrence, and detection scores assigned for each row?	Checks whether every generated row includes the required risk-rating fields.
T10	Are risk scores internally consistent with the described effects, causes, and controls?	Checks whether high-severity effects and weak controls are not given unjustifiably low risk ratings.
T11	Are duplicate or near-duplicate failure modes removed or merged?	Checks whether global review has reduced repeated rows across parts or groups.
T12	Does each row include evidence, rationale, or a link to model context?	Checks whether generated outputs are connected back to source model information or ontology-guided reasoning.

For each test, the generated FMEA is inspected and assigned one of three outcomes: pass, partial pass, or fail. A pass means the requirement is satisfied clearly. A partial pass means the requirement is satisfied for some rows but not consistently across the table. A fail means the requirement is mostly absent or not demonstrated. The verification results are reported in Chapter 4.

3.11.2 Validation Method

The validation step evaluates whether the framework output is useful for the intended purpose of supporting downstream engineering analysis. The validation focuses on four criteria: reliability, traceability, maintainability, and usability. These criteria are used to judge whether the generated analysis is meaningful, reviewable, and useful in the context of the case study.

Reliability examines whether the generated analysis is consistent with the available model information. For example, a failure mode should be relevant to the item or function being analysed, and the cause-effect chain should be plausible in relation to the model context.

Traceability examines whether analysis outputs can be linked back to relevant model context, assumptions, and ontology concepts. A traceable row should make it possible to understand why it was generated and which part of the model supports it.

Maintainability examines whether the analysis can be reviewed, revised, or updated when information changes. This is especially important in Digital Engineering because model information evolves over time. A maintainable analysis should not be a disconnected static document.

Usability examines whether the output is understandable and useful for engineering review. Even if an output is technically structured, it has limited value if engineers cannot interpret it, inspect its rationale, or revise it.

Table 3.6 summarises the validation criteria.

Table 3.6: Validation criteria for the framework demonstration

Criterion	Meaning	Example question
Reliability	The analysis is consistent with the available model information.	Does the failure mode fit the item, function, and context?
Traceability	The analysis output can be linked to model context and ontology-guided reasoning concepts.	Can the row be justified using model information and ontology concepts?
Maintainability	The analysis can be reviewed, revised, or updated after feedback or model changes.	Can the table be revised without rebuilding the analysis manually?
Usability	The output is understandable and useful for engineering review.	Can an engineer inspect, understand, and challenge the result?

The validation performed in this thesis is limited to the case study demonstration. The framework is not tested in a real industrial project, with practising engineers, or in an operational safety-critical environment. Therefore, the validation should be understood as an analytical and case-study-based validation, not as full real-world validation. The results can indicate whether the framework is promising and where it requires improvement, but they cannot prove general industrial validity.

The evaluation also identifies weaknesses in the framework. These weaknesses are used to propose improvements in the discussion chapter.

3.12 Methodological Boundaries

Several boundaries apply to this methodology. First, the framework is demonstrated using a compact toy flashlight model rather than a full industrial Digital Engineering environment. The case study is intentionally limited so that the framework can be inspected clearly within the scope of an MSc thesis.

Second, the parser is designed for the textual SysML-style model used in this thesis. It is not claimed to cover every possible SysML format, every modelling tool export, or the full SysML language specification. Its purpose is to demonstrate how structured model context can be extracted for downstream analysis.

Third, the ontology provides reasoning guidance but does not guarantee complete engineering correctness. It helps organise concepts and relationships, but domain expertise is still required to judge whether a generated failure mode, cause, effect, or score is valid.

Fourth, AI-supported outputs require human review. The methodology demonstrates assisted analysis, not full automation of engineering judgement. This is especially important because FMEA involves interpretation, prioritisation, and risk-related decisions.

Finally, the case study evaluates usefulness, traceability, and maintainability in a controlled demonstration. It does not prove universal industrial validity. Instead, it provides evidence for how the proposed framework can support higher model integrity in a concrete downstream engineering analysis task.

3.13 Summary

This chapter presented the methodology used to develop and demonstrate the model integrity framework. The framework consists of model representation, model parsing, ontology-based reasoning guidance, and AI-supported reasoning. The model representation provides the system-specific engineering information. The parser transforms the raw textual model into structured context for AI use. The ontology provides a domain-agnostic reasoning structure for failure analysis. The AI component combines the parser output and ontology guidance to support generation, review, scoring, explanation, and revision.

The framework is demonstrated using a toy flashlight system model and FMEA as the downstream engineering analysis task. The generated outputs are evaluated according to reliability, traceability, maintainability, and usability. The next chapter applies this methodology to the case study and presents the resulting analysis.

Chapter 3 Draft Outline

This coloured section is a draft outline only. It is intended to guide the structure and logical flow of Chapter 3 and is not part of the final thesis text.

3.1. Research Design

3.1.1. The thesis follows a design-oriented research methodology.

3.1.2. The objective is to develop a framework for improving model integrity in Digital Engineering.

3.1.3. The framework combines model representation, model parsing, ontology-based guidance, and AI-supported reasoning.

3.1.4. The framework is demonstrated using a toy flashlight SysML-style model.

3.1.5. Failure Mode and Effects Analysis (FMEA) is used as the downstream analysis task.

3.1.6. FMEA is used as a demonstration case, not as the main goal of the thesis.

3.2. Framework Overview

3.2.1. The framework contains four main elements: model representation, model parser, ontology, and AI-supported reasoning.

3.2.2. The model representation provides system-specific engineering information.

3.2.3. The model parser extracts bounded and relevant model context.

3.2.4. The ontology provides domain-agnostic reasoning guidance.

3.2.5. The parser tells the AI what to reason about.

3.2.6. The ontology tells the AI how to structure the reasoning.

3.2.7. The AI combines both inputs to support downstream analysis.

3.2.8. Human review keeps the output inspectable, correctable, and accountable.

3.3. Model Representation

3.3.1. The system is represented using a textual SysML-style model.

3.3.2. The toy flashlight model is used as the main case study model.

3.3.3. The model includes requirements, actions, parts, ports, attributes, connections, states, and allocations.

3.3.4. The model is treated as the source of structured analysis context, not only as documentation.

3.3.5. The raw model is too nested and complex to use directly as AI input.

3.3.6. A model parser is therefore needed to transform the model into structured records.

3.4. Model Parser

3.4.1. The parser transforms the raw SysML-style model into structured part-level records.

3.4.2. Each record contains the part name, parent part, layer, original model text, and related context.

3.4.3. The parser avoids giving the AI the full model as one unstructured text block.

3.4.4. The parser uses comment masking, brace matching, declaration parsing, hierarchy extraction, and context extraction.

3.4.5. Related context includes requirements, actions, ports, attributes, connections, bindings, flows, allocations, and states.

3.4.6. The parser provides bounded, system-specific context for AI-supported reasoning.

3.5. Ontology

3.5.1. The ontology is an independent semantic input to the AI component.

3.5.2. The ontology works in parallel with the parser output.

3.5.3. The ontology is developed from FMEA and risk-analysis guidance.

3.5.4. The ontology is formalised as a graph-like JSON structure for GraphRAG-based use.

3.5.5. It contains FMEA concepts such as item, function, failure mode, cause, effect, control, rating, action, and evidence.

3.5.6. It also includes reasoning rules, failure domains, failure-mode categories, cause categories, control methods, and AI question templates.

3.5.7. The ontology guides how the AI structures failure-analysis reasoning.

3.5.8. The full ontology is described in Appendix C.

3.6. AI-Supported Reasoning

3.6.1. The AI component receives two complementary inputs: parsed model context and ontology guidance.

3.6.2. Parsed model context grounds the AI in the specific system being analysed.

3.6.3. Ontology guidance structures the reasoning according to FMEA concepts.

3.6.4. AI supports candidate generation, review, scoring, explanation, and revision.

3.6.5. AI output is not treated as automatically correct.

3.6.6. Human review remains necessary for judgement, interpretation, and accountability.

3.7. Demonstration Through Failure Analysis

3.7.1. The framework is demonstrated through automated failure analysis.

3.7.2. FMEA is selected because it depends strongly on connected engineering information.

3.7.3. The toy flashlight model provides structural and behavioural information for the demonstration.

3.7.4. The demonstration examines whether model information can support traceable FMEA generation.

3.7.5. Weak model integrity can make FMEA incomplete, inconsistent, difficult to justify, or difficult to update.

3.7.6. FMEA provides a useful evaluation surface for the framework.

3.8. Failure Analysis Generation Procedure

3.8.1. Parsed model records are grouped by parent part.

3.8.2. Grouping supports analysis at subsystem or functional-group level.

3.8.3. For each group, the AI receives grouped model context and ontology guidance.

3.8.4. The AI generates candidate FMEA rows.

3.8.5. Group-level review improves relevance, specificity, terminology, and cause-effect consistency.

3.8.6. Global review removes duplicates and normalises terminology across all groups.

3.8.7. Severity, occurrence, and detection scores are assigned using a scoring rubric.

3.8.8. The output is a reviewed and scored FMEA table.

3.9. FMEA Table Structure

3.9.1. Each FMEA row represents one candidate failure mode.

3.9.2. Each row is linked to parsed model context and ontology-guided reasoning.

3.9.3. The table includes item, parent group, function, failure mode, cause, effect, control, scores, RPN, action, and evidence.

3.9.4. RPN is calculated from severity, occurrence, and detection.

3.9.5. The table is structured to support engineering review and revision.

3.10. Interactive Review and Revision

3.10.1. The generated FMEA table is not treated as a final static output.

3.10.2. The user can inspect and revise the table through a chat interface.

3.10.3. Elicitation mode helps the user understand the current table.

3.10.4. Revision mode allows corrections, additions, removals, and refinements.

3.10.5. The interaction keeps human judgement inside the workflow.

3.10.6. This supports reviewability and maintainability of the analysis output.

3.11. Evaluation Method

3.11.1. The framework is evaluated through verification and validation.

3.11.2. Verification checks the generated FMEA against predefined structural and consistency tests.

3.11.3. Verification tests include part coverage, correct hierarchy, function linkage, clear failure modes, cause-effect separation, interface coverage, requirement reflection, score completeness, duplicate removal, and evidence linkage.

3.11.4. Validation assesses reliability, traceability, maintainability, and usability.

3.11.5. Reliability checks consistency with model information.

3.11.6. Traceability checks links back to model context and ontology concepts.

3.11.7. Maintainability checks whether the analysis can be reviewed and updated.

3.11.8. Usability checks whether the output is understandable and useful for engineering review.

3.11.9. Validation is limited to the case study and does not prove real-world industrial validity.

3.12. Methodological Boundaries

3.12.1. The framework is demonstrated using a compact toy flashlight model.

3.12.2. The case study is not a full industrial Digital Engineering environment.

3.12.3. The parser is designed for the textual SysML-style model used in this thesis.

3.12.4. The ontology provides reasoning guidance but does not guarantee engineering correctness.

3.12.5. AI-supported outputs require human review.

3.12.6. The methodology demonstrates assisted analysis, not full automation of engineering judgement.

3.12.7. The evaluation demonstrates usefulness within a controlled case study, not universal industrial validity.

3.13. Summary

3.13.1. The chapter presents the methodology for developing and demonstrating the model integrity framework.

3.13.2. The framework combines model representation, model parsing, ontology-based guidance, and AI-supported reasoning.

3.13.3. The toy flashlight model is used as the demonstration case.

3.13.4. FMEA is used as the downstream analysis task.

3.13.5. The output is evaluated through verification and validation.

3.13.6. Chapter 4 applies the methodology and presents the results.

Chapter 4

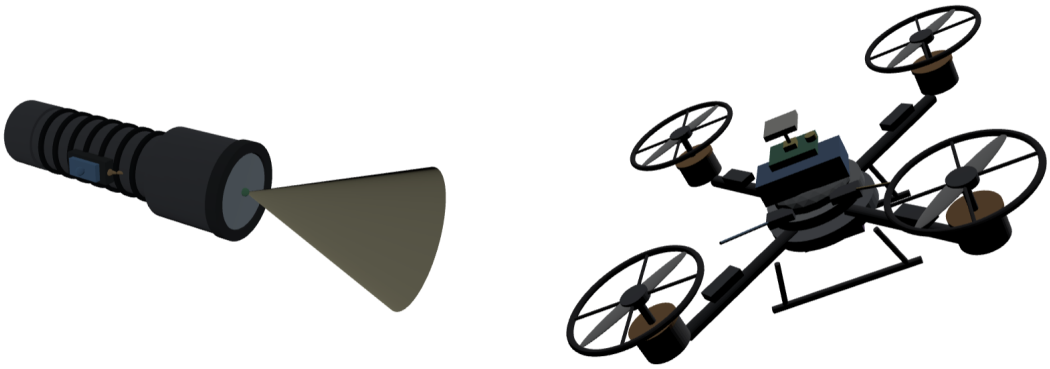
Results

4.1 Case Study Setup

This chapter applies the methodology described in Chapter 3 to two SysML-style case study models. The first case study is the toy flashlight model, which is used as a simple and inspectable baseline case. The second case study is the toy quadcopter model, which is used as a more complex model to examine how the framework behaves when the system contains more parts, interfaces, functions, requirements, and safety-related concerns.

The purpose of using two case studies is to evaluate the proposed model integrity framework across different levels of model complexity. The flashlight model provides a compact example where the relationships between requirements, actions, parts, ports, connections, and allocations can be inspected manually. The quadcopter model provides a richer example with multiple subsystems, repeated components, control logic, telemetry, sensing, propulsion, and safety-related requirements.

Both case studies are used to evaluate whether the framework can support traceable and reviewable Failure Mode and Effects Analysis (FMEA) generation from model-based engineering information. The evaluation is not intended to prove industrial validity. Instead, it examines whether the framework works in controlled engineering examples and identifies where the framework should be improved. Figure 4.1 shows the 3D model representations used for the two case studies. The flashlight model is used as the simple baseline model, while the quadcopter model is used as the more complex model.



(a) Flashlight 3D model **(b) Quadcopter 3D model**

Figure 4.1: 3D representations of the two case study models

Table 4.1 summarises the two case study models.

Table 4.1: Overview of the two case study models

Case study	Purpose	Model characteristics
Flashlight model	Simple baseline case used to test the framework on a compact and manually inspectable system.	Contains requirements, action flow, parts, ports, states, connections, bindings, and requirement allocations.
Quadcopter model	More complex case used to test the framework on a richer multi-subsystem model.	Contains flight-performance, sensing, control, energy, safety, telemetry, propulsion, and physical design information.

The complete flashlight and quadcopter models are provided in Appendix A and Appendix B, respectively.

4.2 Case Study Models

4.2.1 Flashlight Model

The flashlight model represents a compact electromechanical system. It contains a requirements package, an action tree, a parts tree, and a requirement allocation package. The requirements describe expected properties such as field of view, light power, size, weight, reliability, and cost. The action tree describes the functional behaviour required to produce directed light. The main action, `produceDirectedLight`, is decomposed into actions for providing DC power, connecting DC power, generating light, and directing light.

The parts tree represents the physical structure of the flashlight. The main part is `flashlight`, which contains attributes such as mass, field of view, and illumination level. It also contains ports, state behaviour, and internal parts including the battery, switch, lamp, optics, reflector, lens, and structure. Connections and bindings define how power, command, and light-related interactions move through the system.

The flashlight model is suitable as a simple case study because its functional chain is easy to follow: the battery provides power, the switch connects power, the lamp generates light, and the optics direct the light. At the same time, the model includes enough relationships to test the framework, including part hierarchy, actions, ports, connections, states, and requirement allocations.

4.2.2 Quadcopter Model

The quadcopter model represents a more complex multi-subsystem system. It contains requirements related to user interface, flight performance, physical constraints, energy, sensing and control, safety, reliability, and cost. Compared with the flashlight model, the quadcopter model contains a broader set of engineering concerns, including controlled flight, hover stability, flight time, payload, power distribution, telemetry, attitude sensing, altitude sensing, flight control, failsafe behaviour, and low-battery protection.

The action tree describes the behaviour required to produce controlled flight. It includes actions for providing electrical power, distributing electrical power, receiving flight commands, measuring vehicle state, computing flight control, modulating motor power, generating rotor thrust, combining thrust, and reporting telemetry. This creates a more complex functional chain than the flashlight model because sensing, command processing, control computation, power distribution, propulsion, and telemetry are all connected.

The parts tree includes the quadcopter, battery, power distribution board, receiver, flight controller, sensor suite, electronic speed controllers, motors, propellers, thrust mixer, telemetry radio, and frame body. Some parts have multiplicities, such as four electronic speed controllers, four motors, four propellers, and four arms. This makes the model useful for examining how the parser and FMEA generation procedure handle repeated elements and richer interconnections.

Table 4.2 summarises the main differences between the two models.

Table 4.2: Comparison of the flashlight and quadcopter case study models

Aspect	Flashlight model	Quadcopter model
System complexity	Simple compact system.	More complex multi-subsystem system.
Main function	Produce directed light.	Produce controlled flight and telemetry.
Main subsystems	Battery, switch, lamp, optics, structure.	Battery, power distribution, receiver, flight controller, sensor suite, ESCs, motors, propellers, telemetry, frame.
Behavioural content	Power and light-generation action chain.	Power, command, sensing, control, propulsion, thrust, and telemetry action chains.
Requirement types	Illumination and physical requirements.	User interface, flight performance, physical, energy, sensing/control, and safety requirements.
Purpose in evaluation	Baseline inspectable case.	Complexity and scalability stress test.

4.3 Model Parser Results

Both case study models were processed by the SysML-style model parser described in Chapter 3. The purpose of the parser is to transform raw textual model content into structured records that can be used by the AI-supported reasoning component. Each parsed record contains the part name, parent part, hierarchical layer, original part text, and related context.

For the flashlight model, the parser output is expected to be compact and manually inspectable. The key parsed elements include the flashlight, battery, switch, lamp, optics, reflector, lens, structure, and housing elements. The parser also extracts relevant context such as attributes, ports, performed actions, state transitions, bindings, connections, and requirement allocations.

For the quadcopter model, the parser output is more complex. The model includes deeper hierarchy, more ports, more connections, repeated component multiplicities, and more requirement allocations. The parser must therefore handle more model relations and larger context sets. This case is useful for examining whether the parser still produces bounded and meaningful AI input when model complexity increases.

Table 4.3 provides a summary of parser results. The exact values should be completed after the final parser execution.

Table 4.3: Summary of parser results for the two case studies

Parser output measure	Flashlight model	Quadcopter model
Number of parsed part records		
Number of top-level parts		
Number of nested child parts		
Number of extracted ports		
Number of extracted attributes		
Number of extracted connections and bindings		
Number of extracted actions or performed actions		
Number of extracted requirements or allocations		
Maximum hierarchy depth		
Main parser issues observed		

The parser results are important for evaluating the framework because they determine the quality of the context supplied to the AI component. If relevant parts, actions, connections, or allocations are missed, the downstream FMEA may also miss corresponding failure modes or evidence links. Conversely, if the parser preserves relevant context while reducing irrelevant text, the AI receives a more focused and inspectable input.

4.4 Ontology Use in the Case Studies

The same domain-agnostic FMEA ontology was used for both the flashlight and quadcopter case studies. This is important because the ontology is intended to provide general reasoning guidance rather than system-specific content. The model parser provides the system-specific context, while the ontology provides the conceptual structure for failure-analysis reasoning.

In both case studies, the ontology supports the AI by defining concepts such as item, function, failure mode, cause, effect, control, severity, occurrence, detection, recommended action, and evidence. It also provides broader reasoning support through failure domains, failure-mode categories, cause categories, control methods, reasoning rules, and AI question templates. This helps the AI produce FMEA rows that follow a more systematic structure.

For the flashlight model, the ontology helps transform simple functions such as providing DC power, connecting DC power, generating light, and directing light into possible failure modes. For the quadcopter model, the ontology helps reason over a wider set of functions, including power distribution, command reception, sensing, flight control computation, motor power modulation, rotor thrust generation, thrust combination, and telemetry reporting.

The use of the same ontology in both models tests whether the ontology can support different levels of system complexity. The flashlight case tests whether the ontology supports a simple functional chain. The quadcopter case tests whether the ontology remains useful when the model contains more subsystems, interfaces, operating states, and safety-related requirements.

4.5 Generated FMEA Results

The FMEA generation procedure was applied to the parsed model records from both case studies. Parsed records were grouped by parent part, and each group was processed using both the bounded model context and the ontology. The AI component generated candidate FMEA rows for each group. These rows were then reviewed at group level and globally across all groups.

Each generated FMEA row follows the schema defined in Chapter 3. The row includes the analysed item or part, parent group, inferred function, failure mode, cause, effect, existing control, severity, occurrence, detection, RPN, recommended action, and evidence or rationale.

The flashlight FMEA output is expected to be relatively small and easy to inspect. It should include failure modes related to battery power, switch connection, lamp light generation, optics light direction, port connections, and physical requirements. The quadcopter FMEA output is expected to be larger and more complex. It should include failure modes related to power distribution, command input, sensing, control computation, motor power modulation, thrust generation, telemetry, safety functions, and repeated propulsion components.

Table 4.4 summarises the generated FMEA outputs. The exact values should be filled in after the final FMEA generation run.

Table 4.4: Summary of generated FMEA outputs

Output measure	Flashlight model	Quadcopter model
Number of generated raw FMEA rows		
Number of rows after group-level review		
Number of rows after global review		
Number of duplicate rows removed		
Number of rows with explicit evidence or rationale		
Number of rows with complete S/O/D scoring		
Main failure domains represented		
Main issues observed in generated output		

The generated outputs provide the main basis for the verification and validation steps. Verification checks whether the tables satisfy predefined structural and consistency tests. Validation assesses whether the outputs are reliable, traceable, maintainable, and usable for engineering review.

4.6 Comparison Between Simple and Complex Case Studies

The flashlight and quadcopter cases provide different perspectives on the framework. The flashlight case tests whether the framework works on a simple, transparent model where outputs can be inspected manually. The quadcopter case tests whether the same framework remains useful when the model contains more elements, more interfaces, more functions, and more safety-related requirements.

The comparison focuses on three aspects. First, it examines how model complexity affects parsing. The flashlight model has a limited number of parts and relationships, while the quadcopter model includes repeated components, deeper hierarchy, and more interconnections. Second, it examines how model complexity affects AI-supported FMEA generation. More model elements and interfaces create more opportunities for relevant failure modes, but they also increase the risk of missing coverage, duplication, or weak evidence. Third, it examines whether the same domain-agnostic ontology remains useful for both systems.

Table 4.5 summarises the comparison between the two case studies.

Table 4.5: Comparison of framework behaviour across the two case studies

Comparison aspect	Flashlight model	Quadcopter model
Ease of manual inspection		
Parser complexity		
Context extraction quality		
FMEA generation complexity		
Duplicate or overlapping rows		
Requirement-to-analysis linkage		
Interface failure coverage		
Usefulness of ontology guidance		
Overall observation		

The comparison helps identify whether the framework is only suitable for simple models or whether it can also provide useful support for richer systems. The results also help identify which parts of the framework require improvement as model complexity increases.

4.7 Verification Results

The verification step evaluates the generated FMEA tables against the predefined tests defined in Chapter 3. Each test is assessed as pass, partial pass, or fail. A pass means that the test is clearly satisfied. A partial pass means that the test is satisfied for some rows or model elements but not consistently across the whole table. A fail means that the test is mostly absent or not demonstrated.

The purpose of verification is not to prove that every generated failure mode is correct. Rather, it checks whether the FMEA output satisfies basic structural, model-coverage, and reasoning expectations. This is important because an FMEA table can appear complete while still missing important model elements, mixing failure modes with causes or effects, or lacking evidence.

Table 4.6 provides the verification results for both case studies. The result cells should be completed after the final evaluation.

Table 4.6: Verification results for the generated FMEA tables

Test	Verification question	Flashlight result	Quadcopter result	Notes
T1	Are all relevant model parts represented in the FMEA table?			
T2	Are parent-child relationships reflected correctly?			
T3	Are functions linked to the correct items or parts?			
T4	Does each FMEA row contain a clear failure mode?			
T5	Are failure modes separated from causes and effects?			
T6	Does each row include a plausible cause-effect chain?			
T7	Are interface-related failures considered where connections exist?			
T8	Are relevant requirements or constraints reflected in the analysis?			
T9	Are severity, occurrence, and detection scores assigned for each row?			
T10	Are risk scores internally consistent with the described effects, causes, and controls?			
T11	Are duplicate or near-duplicate failure modes removed or merged?			
T12	Does each row include evidence, rationale, or a link to model context?			

The verification results are used to identify where the framework satisfies its expected structural behaviour and where it still requires improvement. For example, weak results on part coverage may suggest parser limitations, while weak results on evidence linkage may suggest that the AI prompt or ontology retrieval process should be improved.

4.8 Validation Results

The validation step evaluates whether the generated FMEA outputs are useful for the intended engineering review purpose. In this thesis, validation is analytical and case-study-based. The framework is not tested in a real industrial project, with practising engineers, or in an operational safety-critical environment. Therefore, the validation results should be

interpreted as evidence of usefulness within controlled examples, not as proof of full real-world validity.

Validation is assessed using four criteria: reliability, traceability, maintainability, and usability. Reliability examines whether generated rows are consistent with the available model information. Traceability examines whether rows can be linked back to model context and ontology concepts. Maintainability examines whether the table can be reviewed, revised, or updated after feedback or model changes. Usability examines whether the output is understandable and useful for engineering review.

Table 4.7 summarises the validation results. The rating cells should be completed after final inspection of the outputs.

Table 4.7: Validation results for the generated FMEA outputs

Validation criterion	Flashlight assessment	Quadcopter assessment	Rationale
Reliability			
Traceability			
Maintainability			
Usability			

The validation results help answer whether the framework produces analysis outputs that are meaningful and useful for engineering review. The comparison between the flashlight and quadcopter cases also indicates how the perceived usefulness changes when the model becomes more complex.

4.9 Interactive Review and Revision Results

The generated FMEA tables were also evaluated through example interactive review and revision scenarios. The purpose of this step is to examine whether the framework keeps human judgement inside the workflow and whether generated outputs can be inspected and improved after initial generation.

Two types of interaction are considered. The first is elicitation, where the user asks questions about the existing FMEA table without changing it. Example elicitation questions include asking why a failure mode was generated, which model element supports a row, why a severity score was assigned, or which rows belong to a specific subsystem. The second type is revision, where the user asks for changes to the FMEA table. Example revision requests include removing duplicate rows, adding missing interface failures, changing scores, rewriting causes, or splitting one broad failure mode into more specific rows.

Table 4.8 summarises the interaction examples. The examples should be completed after the interactive review is performed.

Table 4.8: Interactive review and revision examples

Case study	Interaction type	Example user request	Observed result
Flashlight	Elicitation		
Flashlight	Revision		
Quadcopter	Elicitation		
Quadcopter	Revision		

The interaction results are important because they show whether the generated FMEA table is a static document or a maintainable analysis artefact. A maintainable output should allow the user to inspect the reasoning, challenge weak rows, correct mistakes, and update the table without rebuilding the entire analysis manually.

4.10 Identified Framework Improvements

The verification and validation results are used to identify improvements to the framework. These improvements are grouped into parser improvements, ontology improvements, AI reasoning improvements, and evaluation improvements.

Parser improvements may include better handling of nested model elements, repeated multiplicities, relation extraction, and requirement allocations. This is especially relevant for the quadcopter model, where repeated parts such as motors, propellers, and electronic speed controllers can create ambiguity in how failure modes are represented.

Ontology improvements may include cleaner formalisation, stronger domain-specific extensions, and more explicit evidence concepts. Although the ontology is domain-agnostic, some systems may require domain-specific failure knowledge to produce more precise results. For example, a quadcopter model may benefit from more specific control, propulsion, battery, and sensing failure knowledge.

AI reasoning improvements may include stricter prompts, better evidence binding, clearer uncertainty handling, and improved separation between generated assumptions and model-supported claims. These improvements are important because AI-supported outputs may sound plausible even when they are weakly supported.

Evaluation improvements may include expert review, larger case studies, comparison with human-created FMEA tables, and real-world validation. The current evaluation is useful for examining the framework in controlled examples, but it does not prove industrial validity.

Table 4.9 summarises the identified improvements.

Table 4.9: Identified framework improvements

Improvement area	Observed issue	Possible improvement
Model parser		
Ontology		
AI-supported reasoning		
Evidence linking		
Interactive revision		
Evaluation method		

These improvements provide a bridge from the results chapter to the discussion and conclusion. They show that the framework is not presented as a finished industrial tool, but as a research artefact that demonstrates how model representation, parsing, ontology-based guidance, AI-supported reasoning, and human review can be combined to support model integrity.

4.11 Summary of Results

This chapter applied the proposed framework to two SysML-style case study models. The flashlight model was used as a simple and inspectable baseline case, while the quadcopter

model was used as a more complex case with richer subsystem structure and more engineering relationships.

The model parser transformed both raw textual models into structured context. The domain-agnostic FMEA ontology provided the same reasoning guidance for both case studies. The AI-supported reasoning component used the parser output and ontology guidance to generate, review, score, explain, and revise FMEA tables.

The verification step checked whether the generated FMEA outputs satisfied predefined structural and consistency tests. The validation step assessed reliability, traceability, maintainability, and usability. The interactive review step examined whether human judgement could remain part of the workflow through elicitation and revision. Finally, the results were used to identify improvements to the parser, ontology, AI reasoning process, evidence linking, interaction design, and evaluation method.

The findings from this chapter provide the basis for the final discussion and conclusion in Chapter 5.

Chapter 4 Draft Outline

This section is a draft outline only. It is intended to guide the structure and logical flow of Chapter 4 and is not part of the final thesis text.

4.1. Case Study Setup

4.1.1. This chapter applies the methodology developed in Chapter 3.

4.1.2. Two SysML-style case study models are used: a toy flashlight model and a toy quadcopter model.

4.1.3. The flashlight model is used as a simple baseline case.

4.1.4. The quadcopter model is used as a more complex case to test the framework on a richer system structure.

4.1.5. Both models include requirements, actions, parts, ports, connections, states, and allocations.

4.1.6. The purpose is to evaluate whether the framework can support traceable and reviewable FMEA generation across different levels of model complexity.

4.1.7. The case studies are not intended to prove industrial validity, but to test the framework in controlled engineering examples.

4.2. Case Study Models

4.2.1. The flashlight model represents a compact system with a limited number of parts and functions.

4.2.2. The flashlight model includes batteries, switch, lamp, optics, reflector, lens, and structure.

4.2.3. The flashlight action model describes how directed light is produced.

4.2.4. The flashlight requirements include illumination and physical constraints.

4.2.5. The quadcopter model represents a more complex multi-subsystem system.

4.2.6. The quadcopter model includes battery, power distribution board, receiver, flight controller, sensor suite, ESCs, motors, propellers, thrust mixer, telemetry radio, and frame body.

4.2.7. The quadcopter action model describes controlled flight, including power distribution, sensing, control, propulsion, thrust generation, and telemetry.

4.2.8. The quadcopter requirements include flight performance, sensing and control, energy, safety, physical constraints, and user interface requirements.

4.3. Model Parser Results

4.3.1. Both models are processed by the SysML-style model parser.

4.3.2. The parser identifies relevant parts and parent-child relationships.

4.3.3. Each parsed record contains part name, parent part, layer, original part text, and related context.

4.3.4. For the flashlight model, the parser output is expected to be small and easy to inspect manually.

4.3.5. For the quadcopter model, the parser output is expected to contain more nested parts, repeated multiplicities, and richer interconnections.

4.3.6. Parser results are compared to examine how model complexity affects context extraction.

4.3.7. Parser limitations, missed elements, or ambiguous relations are recorded for later evaluation.

4.4. Ontology Use in the Case Studies

4.4.1. The same domain-agnostic FMEA ontology is used for both case studies.

4.4.2. The ontology provides general FMEA concepts such as item, function, failure mode, cause, effect, control, rating, action, and evidence.

4.4.3. The ontology is independent from the system model and does not change between the flashlight and quadcopter cases.

4.4.4. The parser output tells the AI what system-specific information to consider.

4.4.5. The ontology tells the AI how to structure failure-analysis reasoning.

4.4.6. Using the same ontology across both models tests whether the ontology can support different system complexities.

4.5. Generated FMEA Results

4.5.1. Parsed model records are grouped by parent part.

4.5.2. Candidate FMEA rows are generated for each parent group.

4.5.3. The flashlight FMEA output is used to inspect whether the framework works on a simple, transparent system.

4.5.4. The quadcopter FMEA output is used to inspect whether the framework remains useful on a more complex system.

4.5.5. Generated rows include item, function, failure mode, cause, effect, control, scores, action, and evidence.

4.5.6. Group-level review improves relevance and cause-effect consistency.

4.5.7. Global review removes duplicates and normalises terminology.

4.5.8. The final FMEA tables for both models are presented as the main analysis outputs.

4.6. Comparison Between Simple and Complex Case Studies

4.6.1. The flashlight and quadcopter results are compared to examine the effect of model complexity.

4.6.2. The flashlight case tests basic coverage, traceability, and explainability.

4.6.3. The quadcopter case tests scalability to more parts, more functions, more interfaces, and more safety-related requirements.

4.6.4. The comparison examines whether the parser still extracts useful context as model complexity increases.

4.6.5. The comparison examines whether the ontology remains useful across different system domains.

4.6.6. The comparison examines whether AI-generated FMEA rows become harder to verify and validate as complexity increases.

4.7. Verification Results

4.7.1. The generated FMEA tables are evaluated against the predefined verification tests.

4.7.2. Each test is scored as pass, partial pass, or fail for each case study.

4.7.3. The tests check part coverage, hierarchy, function linkage, failure-mode clarity, and cause-effect separation.

4.7.4. The tests also check interface coverage, requirement reflection, score completeness, duplicate removal, and evidence linkage.

4.7.5. Verification results are reported separately for the flashlight and quadcopter models.

4.7.6. The results are compared to identify which tests become harder to satisfy as model complexity increases.

4.8. Validation Results

4.8.1. Validation evaluates the usefulness of the outputs for engineering review.

4.8.2. Reliability is assessed by checking whether generated rows are consistent with each model context.

4.8.3. Traceability is assessed by checking whether rows can be linked back to model elements and ontology concepts.

4.8.4. Maintainability is assessed by checking whether the tables can be reviewed and revised after feedback or model changes.

4.8.5. Usability is assessed by checking whether the tables are understandable and practical for engineering review.

4.8.6. Validation is reported for both the flashlight and quadcopter cases.

4.8.7. Validation remains analytical and case-study-based, not real-world industrial validation.

4.9. Interactive Review and Revision Results

4.9.1. The generated FMEA tables are tested through example user interactions.

4.9.2. Elicitation examples show how the system explains existing rows, evidence, and scores.

4.9.3. Revision examples show how the system updates rows after user feedback.

4.9.4. The flashlight case is used to demonstrate simple explanation and revision.

4.9.5. The quadcopter case is used to demonstrate explanation and revision in a more complex model context.

4.9.6. The interaction demonstrates how human judgement remains part of the workflow.

4.9.7. The results show whether the framework supports reviewability and maintainability.

4.10. Identified Framework Improvements

4.10.1. The evaluation results are used to identify possible framework improvements.

4.10.2. Parser improvements may include better handling of nested model elements, repeated multiplicities, allocations, and relation extraction.

4.10.3. Ontology improvements may include cleaner formalisation, stronger domain-specific extensions, and improved evidence concepts.

4.10.4. AI reasoning improvements may include stricter prompts, better evidence binding, and stronger uncertainty handling.

4.10.5. Evaluation improvements may include expert review, larger case studies, real-world validation, and comparison with human-created FMEA tables.

4.11. Summary of Results

4.11.1. The chapter applies the proposed framework to two SysML-style case studies.

4.11.2. The flashlight model demonstrates the framework on a simple and inspectable system.

4.11.3. The quadcopter model demonstrates the framework on a richer and more complex system.

4.11.4. The parser transforms both models into structured context.

4.11.5. The ontology provides the same reasoning guidance for both cases.

4.11.6. The AI component generates and refines FMEA tables using both parser output and ontology guidance.

4.11.7. Verification checks whether the outputs satisfy predefined tests.

4.11.8. Validation assesses reliability, traceability, maintainability, and usability.

4.11.9. The results provide the basis for the final discussion and conclusion in Chapter 5.

Chapter 5

Conclusion

This thesis addressed the problem of poor model integrity in Digital Engineering environments. The central motivation was that engineering models are increasingly expected to support downstream analysis, decision-making, verification, and review. However, the existence of models alone does not guarantee that the information inside them is reliable, traceable, current, consistent, or usable. When model information is incomplete, weakly connected, or difficult to inspect, downstream analysis becomes slower, harder to justify, and more difficult to maintain after changes.

To address this problem, the thesis developed a model integrity framework that combines four main elements: model representation, model parsing, ontology-based reasoning guidance, and AI-supported reasoning. The model representation provides the system-specific engineering information. The model parser transforms raw SysML-style model text into structured and bounded context. The ontology provides domain-agnostic reasoning guidance for failure analysis. The AI component then uses both the parser output and the ontology to support the generation, review, scoring, explanation, and revision of downstream analysis outputs.

The framework was demonstrated through two SysML-style case studies. The flashlight model was used as a simple and inspectable baseline case, while the quadcopter model was used as a more complex case with richer subsystem structure, repeated elements, sensing, control, propulsion, safety, and telemetry concerns. Failure Mode and Effects Analysis (FMEA) was used as the downstream analysis task because it depends strongly on connected engineering information and exposes weaknesses in model coverage, traceability, and reasoning structure.

The thesis does not claim that the proposed framework solves all model integrity problems in Digital Engineering. It also does not claim that AI can replace engineering judgement. Instead, the thesis shows how AI can be positioned as one supporting method within a broader model integrity workflow. The main contribution is the structured combination of model-based context, ontology-guided reasoning, AI-supported assistance, and human review.

5.1 Answer to the Research Questions

This thesis was guided by two research questions.

RQ1: How can existing tools and methods, including descriptive modelling notations, semantic representations, and artificial intelligence, be organised into a practical framework for improving model integrity, with Failure Mode and Effects Analysis used as the demonstration case?

The thesis answered this question by proposing a framework in which each method has a distinct role. The SysML-style model representation provides the system-specific engineer-

ing information. The model parser extracts relevant information from the model and turns it into structured context. The ontology provides a domain-agnostic reasoning structure for FMEA. The AI component combines the parsed model context and ontology guidance to support downstream analysis.

This organisation is important because it separates two different types of knowledge. The parser provides what the AI should reason about: the system-specific parts, functions, ports, connections, actions, requirements, and allocations. The ontology provides how the AI should reason: the FMEA concepts, failure logic, cause-effect relations, controls, ratings, evidence, and review questions. This separation reduces the risk of asking the AI to reason directly from one large unstructured model text block.

The framework therefore organises existing methods into complementary roles rather than treating them as competing solutions. Descriptive modelling provides the source information. Parsing makes that information usable. Ontology provides semantic guidance. AI supports selected reasoning tasks. Human review preserves engineering judgement and accountability.

RQ2: To what extent does the proposed framework support the reliability, traceability, and maintainability of downstream engineering analysis, and how can the framework be improved, evaluated through FMEA as a case study?

The framework supports downstream analysis by producing FMEA outputs that are structured around model-derived context and ontology-guided reasoning. Reliability is supported because generated rows are expected to remain consistent with the available model information. Traceability is supported because rows can be connected back to parsed model context and ontology concepts. Maintainability is supported because the generated table can be reviewed, explained, and revised through the interactive workflow.

The verification and validation approach in Chapter 4 was used to assess this support. Verification checked whether the generated FMEA tables satisfied predefined structural and consistency tests, such as model part coverage, hierarchy reflection, function linkage, failure-mode clarity, cause-effect separation, interface coverage, requirement reflection, score completeness, duplicate removal, and evidence linkage. Validation assessed reliability, traceability, maintainability, and usability.

The results also show that the framework can be improved. Parser robustness, evidence linking, ontology formalisation, prompt control, uncertainty handling, and expert validation remain important areas for further development. The framework is therefore best understood as a research-grade demonstration of how model integrity can be supported, not as a finished industrial tool.

5.2 Main Findings

The first main finding is that model representation alone is not sufficient for downstream analysis. A SysML-style model can contain useful engineering information, but this information must be extracted, bounded, and organised before it can reliably support AI-assisted analysis. Without this step, the AI may receive too much irrelevant context or miss important model relationships.

The second main finding is that the model parser plays a central role in improving the usability of model information. By extracting part-level records, parent-child relationships, local context, actions, ports, connections, bindings, requirements, and allocations, the parser transforms raw model text into a more analysable structure. This supports model integrity because the information used for downstream reasoning becomes more explicit and inspectable.

The third main finding is that the ontology provides a necessary reasoning frame. The ontology does not replace the system model. Instead, it provides domain-agnostic FMEA concepts and relationships that guide how the AI structures its analysis. This distinction is

important: the parser grounds the AI in the system, while the ontology guides the reasoning process.

The fourth main finding is that FMEA is a useful demonstration task for model integrity. FMEA requires functions, items, causes, effects, controls, ratings, actions, and evidence. Because of this, weak model integrity becomes visible quickly. Missing model elements can lead to missing failure modes. Weak connections can lead to unsupported effects. Poor traceability can make scores and recommended actions difficult to justify.

The fifth main finding is that model complexity increases the challenge. The flashlight model is compact and easier to inspect manually. The quadcopter model introduces more parts, repeated components, more interfaces, more safety-related requirements, and more functional dependencies. This makes parsing, coverage checking, duplicate removal, and evidence linking more difficult. The comparison between the two models therefore shows why model integrity methods must be robust to increasing system complexity.

The final finding is that human review remains necessary. AI-supported reasoning can help generate, review, explain, and revise analysis outputs, but it cannot be treated as automatically correct. Engineering analysis still requires judgement, accountability, and domain expertise.

5.3 Contributions

This thesis makes several contributions.

First, it proposes a practical framework for improving model integrity in Digital Engineering environments. The framework combines model representation, model parsing, ontology-based reasoning guidance, AI-supported reasoning, and human review.

Second, it clarifies the different roles of the parser and the ontology. The parser provides bounded system-specific context, while the ontology provides domain-agnostic reasoning structure. This distinction helps avoid the common mistake of treating AI as if it can reason reliably from raw model text alone.

Third, the thesis develops and presents a SysML-style model parser for extracting structured part-level context from textual models. The parser provides a bridge between model representation and AI-supported downstream analysis.

Fourth, the thesis uses a domain-agnostic FMEA ontology as a semantic reasoning guide. The ontology includes FMEA concepts, reasoning rules, failure domains, failure-mode categories, cause categories, control methods, rating concepts, evidence concepts, and AI question templates.

Fifth, the thesis demonstrates the framework using two case studies: a simple flashlight model and a more complex quadcopter model. This allows the framework to be examined across different levels of model complexity.

Sixth, the thesis proposes a verification and validation approach for assessing AI-supported FMEA outputs. Verification is based on predefined structural and consistency tests, while validation uses reliability, traceability, maintainability, and usability as evaluation criteria.

5.4 Implications for Digital Engineering

The findings have several implications for Digital Engineering.

First, Digital Engineering requires more than digitised documents or isolated models. The value of a Digital Engineering environment depends on whether model information can be connected, interpreted, reused, and maintained over time. Model integrity is therefore not a minor modelling concern. It is central to whether models can support downstream analysis and decision-making.

Second, AI should not be treated as a shortcut around model quality. If the model information is incomplete, ambiguous, or weakly connected, AI may produce outputs that appear convincing but are poorly grounded. This thesis supports the view that AI is most useful when it is bounded by structured model context, guided by explicit semantic reasoning, linked to evidence, and reviewed by humans.

Third, ontologies and graph-based representations can help make reasoning structures more explicit. They are especially useful when AI systems need to reason consistently across concepts such as function, failure mode, cause, effect, control, risk, action, and evidence. However, ontologies also require maintenance and governance. They are not magic dust sprinkled on a model to make it smart. Sadly, thesis dragons still require actual engineering.

Fourth, FMEA is a useful evaluation surface for model integrity because it turns hidden model weaknesses into visible analysis problems. If functions, interfaces, requirements, or evidence links are missing, the FMEA table will often expose those weaknesses through incomplete rows, unsupported causes, weak effects, or unclear scores.

5.5 Limitations

This thesis has several limitations.

First, the framework was demonstrated only on toy SysML-style models. The flashlight and quadcopter models are useful for controlled evaluation, but they do not represent the full complexity of industrial Digital Engineering environments.

Second, the parser is designed for the textual SysML-style model structure used in this thesis. It is not claimed to support every possible SysML syntax, SysML v2 implementation, modelling convention, or tool export format. More robust parsing would be required for industrial deployment.

Third, the ontology is domain-agnostic. This makes it reusable across different systems, but it also means that it does not contain detailed domain-specific failure physics. For example, the quadcopter case could benefit from more specialised knowledge about batteries, flight control, propulsion, sensors, and safety-critical control systems.

Fourth, AI-supported outputs are not guaranteed to be correct. The AI can support generation, review, explanation, scoring, and revision, but its outputs still require human review. This is especially important for FMEA because risk assessment involves engineering judgement.

Fifth, the validation in this thesis is analytical and case-study-based. The framework was not validated in a real industrial project, with practising engineers, or in an operational safety-critical environment. Therefore, the results should be interpreted as evidence that the framework is promising, not as proof of general industrial validity.

Finally, the evaluation does not compare the generated FMEA tables against a full expert-created reference FMEA. Such a comparison would be valuable for assessing completeness, correctness, and practical usefulness more rigorously.

5.6 Future Work

Several directions for future work follow from these limitations.

First, the framework should be tested on larger and more realistic industrial models. This would help evaluate whether the approach remains useful when models contain many requirements, subsystems, variants, interfaces, simulations, and verification artefacts.

Second, the model parser should be extended to handle more complete SysML v2 syntax and different modelling tool exports. A stronger parser could support more robust extraction of actions, states, allocations, constraints, ports, flows, and requirements.

Third, the ontology should be refined into a cleaner and more formal representation. Future work could distinguish more clearly between classes, instances, relations, rules, question templates, and evidence types. A more formal ontology could also support stronger consistency checking.

Fourth, domain-specific ontology extensions should be developed. For example, separate extensions could be created for electrical systems, control systems, software-intensive systems, autonomous systems, and safety-critical systems. These extensions would allow the framework to combine domain-agnostic reasoning with domain-specific failure knowledge.

Fifth, evidence linking should be strengthened. Future versions of the framework should connect each generated FMEA row to specific model elements, assumptions, requirements, test results, simulation outputs, or expert rationale. This would improve traceability and reduce unsupported claims.

Sixth, the AI prompting and retrieval strategy should be improved. Future work should investigate stricter prompts, better GraphRAG retrieval, uncertainty marking, assumption tracking, and automated checks for unsupported statements.

Seventh, the framework should be evaluated with domain experts or practising engineers. Expert review would help assess whether the generated outputs are useful, understandable, and trustworthy in real engineering workflows.

Eighth, future work should compare framework-generated FMEA tables against human-created FMEA tables. This would allow a more rigorous assessment of completeness, quality, effort reduction, and practical value.

Finally, the framework could be extended beyond FMEA. The same model integrity approach may support other downstream engineering tasks, such as verification planning, impact analysis, compliance checking, trade-off analysis, safety analysis, and design review.

5.7 Final Reflection

This thesis shows that improving model integrity requires coordination between modelling, semantic structure, AI support, and human judgement. A model alone is not enough. An ontology alone is not enough. AI alone is definitely not enough; that would be like giving a calculator a wizard hat and calling it engineering.

The proposed framework demonstrates a practical way to make model-based information more structured, bounded, explainable, and reviewable for downstream analysis. By combining model representation, parsing, ontology-based guidance, AI-supported reasoning, and human review, the framework provides a step toward more trustworthy use of model information in Digital Engineering.

The main lesson is that AI can support model integrity only when it is grounded in structured model context and guided by explicit reasoning structures. In this thesis, FMEA served as the demonstration task, but the broader contribution is the framework logic: system-specific model context tells the AI what to reason about, while the ontology tells it how to reason. This separation provides a foundation for more reliable, traceable, maintainable, and usable downstream engineering analysis.

Chapter 5 Draft Outline

This coloured section is a draft outline only. It is intended to guide the structure and logical flow of Chapter 5 and is not part of the final thesis text.

5.1. Conclusion

5.1.1. This chapter concludes the thesis and reflects on the proposed model integrity framework.

5.1.2. The thesis addressed the problem of poor model integrity in Digital Engineering environments.

5.1.3. The proposed framework combined model representation, model parsing, ontology-based guidance, AI-supported reasoning, and human review.

5.1.4. The framework was demonstrated through two SysML-style case studies: the flashlight model and the quadcopter model.

5.1.5. Failure Mode and Effects Analysis (FMEA) was used as the downstream analysis task for evaluating the framework.

5.2. Answer to the Research Questions

5.2.1. The first research question asked how existing tools and methods can be organised into a framework for improving model integrity.

5.2.2. The thesis showed that model representation, parsing, ontology-based reasoning guidance, and AI-supported reasoning can be organised into complementary roles.

5.2.3. The model parser provides bounded system-specific context.

5.2.4. The ontology provides domain-agnostic reasoning structure.

5.2.5. The AI component combines both inputs to support downstream analysis generation, review, scoring, explanation, and revision.

5.2.6. The second research question asked how the framework supports reliable, traceable, maintainable, and usable downstream analysis.

5.2.7. The case studies showed that the framework can support structured and reviewable FMEA generation.

5.2.8. The results also showed that the framework still requires improvement, especially in parser robustness, evidence linking, and validation with real users.

5.3. Main Findings

5.3.1. Model representation alone is not sufficient for downstream analysis.

5.3.2. The parser is important because it transforms raw model text into bounded context for AI use.

5.3.3. The ontology is important because it guides how AI structures failure-analysis reasoning.

5.3.4. Separating parser output from ontology guidance improves clarity in the framework.

5.3.5. The flashlight case showed that the framework can work on a simple and inspectable model.

5.3.6. The quadcopter case showed that model complexity increases parsing, coverage, and verification challenges.

5.3.7. Verification tests helped identify whether generated FMEA outputs satisfied expected structural and consistency requirements.

5.3.8. Validation criteria helped assess reliability, traceability, maintainability, and usability.

5.3.9. Human review remains necessary because AI-supported outputs are not automatically correct.

5.4. Contributions

5.4.1. The thesis proposed a practical framework for improving model integrity in Digital Engineering environments.

5.4.2. The thesis clarified the roles of model representation, model parsing, ontology, AI-supported reasoning, and human review.

5.4.3. The thesis developed a SysML-style model parser for extracting structured part-level context.

5.4.4. The thesis used a domain-agnostic FMEA ontology as a reasoning guide for AI-supported analysis.

5.4.5. The thesis demonstrated the framework on both a simple and a more complex SysML-style model.

5.4.6. The thesis proposed verification tests for checking generated FMEA outputs.

5.4.7. The thesis provided an evaluation structure based on reliability, traceability, maintainability, and usability.

5.5. Implications for Digital Engineering

5.5.1. Digital Engineering requires more than creating models; it requires maintaining usable and trustworthy model information.

5.5.2. Model integrity can be improved when system-specific information is structured before being used in downstream analysis.

5.5.3. Semantic guidance can help make AI-supported reasoning more consistent and inspectable.

5.5.4. AI is most useful when bounded by model context, ontology guidance, evidence links, and human review.

5.5.5. FMEA is a useful demonstration task because it exposes weaknesses in model coverage, traceability, and reasoning structure.

5.5.6. The framework supports the idea that AI should be treated as one supporting method within a broader Digital Engineering workflow.

5.6. Limitations

5.6.1. The framework was demonstrated only on toy SysML-style models.

5.6.2. The case studies do not represent full industrial Digital Engineering environments.

5.6.3. The parser is designed for the textual model structure used in this thesis and may not generalise to all SysML exports.

5.6.4. The ontology is domain-agnostic and does not replace domain-specific failure knowledge.

5.6.5. The AI-generated FMEA outputs require human review and cannot be treated as automatically correct.

5.6.6. The validation was analytical and case-study-based, not real-world industrial validation.

5.6.7. No practising engineering team was used to validate the usefulness of the framework in an operational setting.

5.7. Future Work

5.7.1. Future work should test the framework on larger and more realistic industrial models.

5.7.2. The parser should be extended to handle more complete SysML v2 syntax and different tool export formats.

5.7.3. The ontology should be refined into a cleaner and more formal representation.

5.7.4. Domain-specific ontology extensions should be developed for areas such as electrical systems, control systems, software, and safety-critical systems.

5.7.5. Evidence linking should be strengthened so that every generated analysis row can be traced to model elements, assumptions, and supporting artefacts.

5.7.6. The AI prompting and retrieval strategy should be improved to reduce unsupported claims and better handle uncertainty.

5.7.7. Future validation should involve domain experts or practising engineers.

5.7.8. Future work should compare the framework output against human-created FMEA tables.

5.7.9. The framework could be extended beyond FMEA to other downstream analysis tasks such as verification planning, impact analysis, compliance checking, and trade-off analysis.

5.8. Final Reflection

5.8.1. The thesis shows that improving model integrity requires coordination between modelling, semantic structure, AI support, and human judgement.

5.8.2. The framework does not solve all Digital Engineering integrity problems.

5.8.3. However, it demonstrates a practical way to make model-based information more structured, bounded, explainable, and reviewable for downstream analysis.

5.8.4. The main lesson is that AI can support model integrity only when it is grounded in structured model context and guided by explicit reasoning structures.

Bibliography

[1] João Paulo A. Almeida, Luís Ferreira Pires, Giancarlo Guizzardi, and Gerd Wagner. An analysis of the semantic foundation of KerML and SysML v2. In *Conceptual Modeling*, pages 133–151. Springer, Cham, 2024. 13

[2] Manas Bajaj, Sanford Friedenthal, and Ed Seidewitz. Systems modeling language SysML v2 support for digital engineering. *INSIGHT*, 25(1):19–24, 2022. 13

[3] Mary Bone, Mark Blackburn, Benjamin Kruse, John Dzielski, Thomas Hagedorn, and Ian Grosse. Toward an interoperability and integration framework to enable the digital thread. *Systems*, 6(4):46, 2018. 15

[4] Robert Buchmann, Johann Eder, Hans-Georg Fill, Ulrich Frank, Dimitris Karagiannis, Emanuele Laurenzi, John Mylopoulos, Dimitris Plexousakis, and Maribel Yasmina Santos. Large language models: Expectations for semantics-driven systems engineering. *Data & Knowledge Engineering*, 152:102324, 2024. 17, 18, 20

[5] Pierre de Saqui-Sannes, Rob A. Vingerhoeds, Christophe Garion, and Xavier Thirioux. A taxonomy of MBSE approaches by languages, tools and methods. *IEEE Access*, 10:120936–120950, 2022. 9, 12, 13, 20

[6] Daniel Dunbar, Thomas Hagedorn, Mark Blackburn, John Dzielski, Steven Hespelt, Benjamin Kruse, Dinesh Verma, and Zhongyuan Yu. Driving digital engineering integration and interoperability through semantic integration of models with ontologies. *Systems Engineering*, 26(4):365–378, 2023. 14, 16, 20

[7] Daniel Dunbar, Thomas Hagedorn, Mark Blackburn, and Dinesh Verma. A three-pronged verification approach to higher-level verification using graph data structures. *Systems*, 12(1):27, 2024. 15, 16, 20

[8] Christian Ellsel, Andreas Vogelsang, Daniel Mendez, Henning Femmer, and Eric Knauss. Advancing requirements engineering with large language models. *Procedia Computer Science*, 2025. Publisher metadata indicates an article on LLM-supported requirement reformulation and quality assessment in practically relevant workflows. 17

[9] Jeff A. Estefan. Survey of model-based systems engineering methodologies. Technical report, INCOSE, 2008. Rev. B. 9, 12, 20

[10] Alessio Ferrari et al. Formal requirements engineering and large language models: A two-way roadmap. *Information and Software Technology*, 177:107697, 2025. 17, 20

[11] Sanford Friedenthal, Alan Moore, and Rick Steiner. *Systems Modeling Language (SysML) Tutorial*. Object Management Group and International Council on Systems Engineering, 2008. Revision B, tutorial dated 15 April 2008. 13

[12] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, June 1993. 16, 20

[13] Nicola Guarino, Daniel Oberle, and Steffen Staab. What is an ontology? In Steffen Staab and Rudi Studer, editors, *Handbook on Ontologies*, pages 1–17. Springer, Berlin, Heidelberg, 2009. 16, 20

[14] Thomas D. Jr. Hedberg, Manas Bajaj, and Jaime A. Camelio. Using graphs to link data across the product lifecycle for enabling smart manufacturing digital threads. *Journal of Computing and Information Science in Engineering*, 20(1):011011, 2020. 10, 16

[15] Melinda Hodkiewicz, Gloria Ho, Christiane Howard, Suzanne Robinson, and Thomas Kelly. An ontology for reasoning over engineering textual data stored in fmea spreadsheet tables. *Information Processing & Management*, 58(6):102701, 2021. 19

[16] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021. 16, 20

[17] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Computing Surveys*, 2025. 17, 20

[18] Zhaofeng Huang, Robert Hansen, and Zhaofeng Carl Huang. Toward fmea and mbse integration. In *Proceedings of the 2018 Annual Reliability and Maintainability Symposium*, pages 1–7, 2018. 19

[19] Zhaofeng Huang, Steve Swalgen, Heidi Davidz, and John Murray. MBSE-assisted FMEA approach—challenges and opportunities. In *Proceedings of the Annual Reliability and Maintainability Symposium*, pages 1–8, 2017. 19

[20] Tomas Huldt and Ivan Stenius. State-of-practice survey of model-based systems engineering. *Systems Engineering*, 22(2):134–145, 2019. 12, 20

[21] International Electrotechnical Commission. IEC 60812:2018 failure modes and effects analysis FMEA and FMECA, 2018. 19, 20

[22] Artjoms Korsunovs, Irem Y. Tumer, Felician Campean, Jakub Burek, and Michael Henshaw. Towards a model-based systems engineering approach for robotic manufacturing process modelling with automatic fmea generation. In *Proceedings of the Design Society*, volume 2, pages 853–862, 2022. 19

[23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. 18, 20

[24] Junda Ma, Guoxin Wang, Jinzhi Lu, Hans Vangheluwe, Dimitris Kiritsis, and Yan Yan. Systematic literature review of MBSE tool-chains. *Applied Sciences*, 12(7):3431, 2022. 12, 20

[25] Azad M. Madni and Michael Sievers. Model-based systems engineering: Motivation, current status, and research opportunities. *Systems Engineering*, 21(3):172–190, 2018. 1, 9, 12, 13, 14, 20

[26] Natalya F. Noy and Deborah L. McGuinness. Ontology development 101: A guide to creating your first ontology. Technical Report KSL-01-05, Stanford Knowledge Systems Laboratory, 2001. 16

[27] Object Management Group. OMG Systems Modeling Language (SysML) Version 1.7. Formal specification, June 2024. 12, 13, 20

[28] Object Management Group. Kernel modeling language KerML version 1.0 specification. Technical report, Object Management Group, Needham, MA, 2025. 13, 20

[29] Object Management Group. OMG systems modeling language SysML version 2.0 specification. Technical report, Object Management Group, Needham, MA, 2025. 12, 13, 20

[30] Object Management Group. Systems Modeling API and Services Version 1.0. Formal specification, September 2025. 13, 20

[31] Victor Singh and Karen E. Willcox. Engineering design with digital thread. *AIAA Journal*, 56(11):4515–4528, 2018. 9, 10, 20

[32] Claudio Spreafico, Davide Russo, and Catia Rizzi. A state-of-the-art review of FMEA/FMECA including patents. *Computer Science Review*, 25:19–28, 2017. 19, 20

[33] Fei Tao, Xiaoyu Ma, Wei Liu, and Cheng Zhang. Digital engineering: State-of-the-art and perspectives. *Digital Engineering*, 1:100007, 2024. 1

[34] U.S. Department of Defense. MIL-STD-1629A: Procedures for performing a failure mode, effects and criticality analysis. Technical report, Department of Defense, Washington, DC, 1980. 19, 20

[35] U.S. Department of Defense. Digital engineering strategy. Technical report, Department of Defense, Washington, DC, 2018. 1, 9, 20

[36] U.S. Department of Defense. DoD Data Strategy: Unleashing Data to Advance the National Defense Strategy. Technical report, U.S. Department of Defense, October 2020. 10, 14

[37] U.S. Department of Defense. DoD Instruction 5000.97: Digital Engineering. Technical report, Department of Defense, December 2023. Issued 21 December 2023. 9, 10, 14, 15, 20

[38] Haytham Younus, Felician Campean, Sohag Kabir, Pascal Bonnaud, David Delaux, and Unal Yildirim. An ontological framework for the integration of system design and FMEA. *AI EDAM*, 2025. Advance online / issue details may vary. 19

[39] Haytham Younus, Sohag Kabir, Felician Campean, Pascal Bonnaud, and David Delaux. AI- and ontology-based enhancements to FMEA for advanced systems engineering: Current developments and future directions. *Applied Sciences*, 16(5):2464, 2026. 19

Appendix A

Toy Flashlight SysML Model

This appendix provides the textual SysML-style toy flashlight model used as the demonstration case in this thesis. The model includes requirements, actions, parts, ports, attributes, connections, states, and requirement allocations. It is used as the model representation input for the methodology described in Chapter 3.

```
package FlashlightSpecificationAndDesign {
    public import ScalarValues::*;
    public import SI::*;
    public import USCustomaryUnits::*;

    // create a Requirements package that contains the hierarchy of requirements that compose the
     flashlight specification
    package Requirements{
        requirement flashlightSpecification{
            requirement userInterface;
            requirement illumination{
                // the following is a simple text-based requirement
                requirement fieldOfView{
                    doc /* The flashlight field of view shall be 0 - 20 degrees.*/
                }
                requirement lightPower{
                    doc /* The light power shall be a minumum of TBD lumens.*/
                }
            }
            requirement physical{
                requirement portability;
                requirement size{
                    doc /* The flashlight shall be less than 6 inches in length.*/
                }
                // the following is a more precisely specified requirement using constraints
                requirement weight{
                    doc /* The weight shall be less than 0.25 lbs.*/
                    attribute actualWeight :> ISQ::mass;
                    require constraint {actualWeight <= 0.25 [lb]}
                }
            }
            requirement reliability;
            requirement cost;
        }
    }

    // create a package called ActionTree that contains the actions and flows to produceDirectedLight
    package ActionTree{
        action produceDirectedLight{
            in item onOffCmd; //input
            // the following output (i.e., lightOut) from produceDirectedLight is equal to (i.e.,
     bound to) the lightOut from the action directLight;
            out item lightOut = directLight.lightOut;

            // declare each nested action and their inputs and outputs
            action provideDCPwr{
                out item dcPwr;
            }
            action connectDCPwr {
                in item onOffCmd = produceDirectedLight.onOffCmd;
                in item dcPwrIn;
```

```
            out item dcPwrOut;
        }
        action generateLight{
            in item dcPwrIn;
            out item light;
        }
        action directLight{
            in item lightIn;
            out item lightOut;
        }

        // the following declares a fork node named fork1 that is succeeded by two parallel
 actions
        fork fork1;
            then provideDCPwr; // succession
            then connectDCPwr; // succession

        // define a control flow beginning with a start node and then fork1
        first start then fork1;

        // define input/output flow from the output of one action to the input of another
        flow from provideDCPwr.dcPwr to connectDCPwr.dcPwrIn;
        flow from connectDCPwr.dcPwrOut to generateLight.dcPwrIn;
        flow from generateLight.light to directLight.lightIn;
    }
}

// create a PartsTree package that contains the flashlight,
// along with its key features such as attributes, ports, actions, parts, and interconnections
package PartsTree{
    part flashlight {
        // flashlight attributes
        attribute mass :> ISQ::mass;
        attribute fov:Real;
        attribute illuminationLevel:Real;

        // flashlight ports
        port cmdPort;
        port lightOutPort;

        // flashlight perform action. The flashlight performs the action called
 produceDirectedLight
        perform ActionTree::produceDirectedLight;

        // the flashlight exhibits states: initial, off, and on
        exhibit state flashlightStates{
            // declare the states
            state initial;
            state off;
            state on{
                // define the do action by referring to the action above
                do produceDirectedLight;
            }

            // each state transition specifies start and end states
            transition initial then off;
            transition off_To_on
                first off
                then on;
            transition on_To_off
                first on
                then off;
        }

        // the battery is a referential part of the flashlight
        ref part battery [2]{
            attribute power:> ISQ::power;
            port dcPwrOutPort;
            perform produceDirectedLight.provideDCPwr;
        }

        // the following are composite parts of the flashlight
        part switch{
            port cmdPort;
            port inPort;
            port outPort;
            perform produceDirectedLight.connectDCPwr;
        }

        part lamp{
```

```
                attribute efficiency:Real;
                port dcPwrInPort;
                port lightOutPort;
                perform produceDirectedLight.generateLight;
            }

            part optics{
                port lightInPort;
                port lightOutPort;
                perform produceDirectedLight.directLight;

                part reflector{
                    attribute radius:> ISQ::length;
                }

                part lens;
            }

            part structure{
                part frontHousing;
                part middleHousing;
                part backHousing;
            }

            // binding connections
            bind cmdPort = switch.cmdPort;
            bind lightOutPort = optics.lightOutPort;

            // port connections
            connect battery.dcPwrOutPort to switch.inPort;
            connect switch.outPort to lamp.dcPwrInPort;
            connect lamp.lightOutPort to optics.lightInPort;
        }
    }

    // create a RequirementsAllocation package that contains the allocation of requirements to design
      features
    package RequirementsAllocation{
        public import Requirements::*;
        public import PartsTree::*;

        allocate flashlightSpecification to flashlight{
            allocate flashlightSpecification.physical.weight to flashlight.mass;
            allocate flashlightSpecification.illumination.fieldOfView to flashlight.fov;
            allocate flashlightSpecification.illumination.lightPower to flashlight.illuminationLevel;
        }
    }
}
```

Listing A.1: Toy flashlight SysML-style model used in the case study

Appendix B

Toy Quadcopter SysML Model

This appendix provides the textual SysML-style toy quadcopter model used as an additional demonstration model. The model includes requirements, actions, parts, ports, attributes, connections, states, and requirement allocations. It can be used to test whether the proposed framework scales from a simple flashlight system to a richer multi-subsystem model.

```
package QuadcopterSpecificationAndDesign {
    public import ScalarValues::*;
    public import SI::*;
    public import USCustomaryUnits::*;

    // create a Requirements package that contains the hierarchy of requirements
    // that compose the quadcopter specification
    package Requirements{
        requirement quadcopterSpecification{
            requirement userInterface{
                requirement commandInput{
                    doc /* The quadcopter shall accept pilot or autonomous flight commands through a
    command input interface.*/
                }
                requirement telemetry{
                    doc /* The quadcopter shall provide telemetry including battery state, attitude,
    altitude, and flight mode.*/
                }
            }

            requirement flightPerformance{
                requirement controlledFlight{
                    doc /* The quadcopter shall produce controlled vertical lift and directional
    motion.*/
                }
                requirement hover{
                    doc /* The quadcopter shall maintain stable hover in nominal operating conditions
    .*/
                }
                requirement flightTime{
                    doc /* The quadcopter shall operate for a minimum of TBD minutes on a fully
    charged battery.*/
                    attribute actualFlightTime :> ISQ::time;
                    require constraint {actualFlightTime >= 10 [min]}
                }
                requirement payload{
                    doc /* The quadcopter shall carry a payload of at least TBD mass.*/
                    attribute actualPayloadMass :> ISQ::mass;
                    require constraint {actualPayloadMass >= 0.10 [kg]}
                }
            }

            requirement physical{
                requirement portability;
                requirement size{
                    doc /* The quadcopter shall have a maximum diagonal motor-to-motor size of less
    than 45 centimeters.*/
                    attribute actualDiagonalSize :> ISQ::length;
                    require constraint {actualDiagonalSize <= 45 [cm]}
                }
                requirement weight{
```

```
                doc /* The quadcopter takeoff mass shall be less than 2.0 kg.*/
                attribute actualMass :> ISQ::mass;
                require constraint {actualMass <= 2.0 [kg]}
            }
        }

        requirement energy{
            requirement batteryVoltage{
                doc /* The quadcopter shall use a battery pack with nominal voltage suitable for
 the motor and ESC system.*/
            }
            requirement powerDistribution{
                doc /* The quadcopter shall distribute electrical power from the battery to all
 propulsion and control components.*/
            }
        }

        requirement sensingAndControl{
            requirement attitudeSensing{
                doc /* The quadcopter shall sense roll, pitch, and yaw attitude for flight
 stabilization.*/
            }
            requirement altitudeSensing{
                doc /* The quadcopter shall estimate altitude for takeoff, landing, and hover
 control.*/
            }
            requirement flightControl{
                doc /* The quadcopter shall compute motor commands from desired motion and sensed
 vehicle state.*/
            }
        }

        requirement safety{
            requirement failsafe{
                doc /* The quadcopter shall enter a safe state when communication, power, or
 critical sensor faults are detected.*/
            }
            requirement lowBatteryProtection{
                doc /* The quadcopter shall warn the operator or initiate landing when battery
 state is below a defined threshold.*/
            }
            requirement propellerGuarding{
                doc /* The quadcopter should reduce risk of injury from rotating propellers where
 practical.*/
            }
        }

        requirement reliability;
        requirement cost;
    }
}

// create a package called ActionTree that contains the actions and flows
// required to produce controlled flight
package ActionTree{
    action produceControlledFlight{
        in item flightCmd;
        out item telemetryOut;
        out item thrustOut = combineThrust.thrustOut;

        action provideElectricalPower{
            out item dcPwr;
        }

        action distributeElectricalPower{
            in item dcPwrIn;
            out item propulsionPwr;
            out item controlPwr;
        }

        action receiveFlightCommand{
            in item flightCmdIn = produceControlledFlight.flightCmd;
            out item desiredMotion;
        }

        action measureVehicleState{
            in item controlPwrIn;
            out item sensedState;
        }
```

```
        action computeFlightControl{
            in item desiredMotionIn;
            in item sensedStateIn;
            in item controlPwrIn;
            out item motorCmd;
            out item telemetry;
        }

        action modulateMotorPower{
            in item motorCmdIn;
            in item propulsionPwrIn;
            out item motorPwr;
        }

        action generateRotorThrust{
            in item motorPwrIn;
            out item rotorThrust;
        }

        action combineThrust{
            in item rotorThrustIn;
            out item thrustOut;
        }

        action reportTelemetry{
            in item telemetryIn = computeFlightControl.telemetry;
            out item telemetryOut = produceControlledFlight.telemetryOut;
        }

        fork fork1;
            then provideElectricalPower;
            then receiveFlightCommand;

        first start then fork1;

        flow from provideElectricalPower.dcPwr to distributeElectricalPower.dcPwrIn;
        flow from distributeElectricalPower.controlPwr to measureVehicleState.controlPwrIn;
        flow from distributeElectricalPower.controlPwr to computeFlightControl.controlPwrIn;
        flow from distributeElectricalPower.propulsionPwr to modulateMotorPower.propulsionPwrIn;

        flow from receiveFlightCommand.desiredMotion to computeFlightControl.desiredMotionIn;
        flow from measureVehicleState.sensedState to computeFlightControl.sensedStateIn;

        flow from computeFlightControl.motorCmd to modulateMotorPower.motorCmdIn;
        flow from modulateMotorPower.motorPwr to generateRotorThrust.motorPwrIn;
        flow from generateRotorThrust.rotorThrust to combineThrust.rotorThrustIn;

        flow from computeFlightControl.telemetry to reportTelemetry.telemetryIn;
    }
}

// create a PartsTree package that contains the quadcopter,
// its key features such as attributes, ports, and actions,
// its parts, and its part interconnections
package PartsTree{
    part quadcopter {
        // quadcopter attributes
        attribute mass :> ISQ::mass;
        attribute diagonalSize :> ISQ::length;
        attribute payloadMass :> ISQ::mass;
        attribute flightTime :> ISQ::time;
        attribute batteryStateOfCharge : Real;
        attribute hoverStability : Real;

        // quadcopter ports
        port cmdPort;
        port telemetryPort;
        port thrustOutPort;

        perform ActionTree::produceControlledFlight;

        // the quadcopter exhibits states for basic operating modes
        exhibit state quadcopterStates{
            state initial;
            state off;
            state armed;
            state flying{
                do produceControlledFlight;
            }
            state landing{
```

```
            do produceControlledFlight;
        }
        state fault;

        transition initial then off;

        transition off_To_armed
            first off
            then armed;

        transition armed_To_flying
            first armed
            then flying;

        transition flying_To_landing
            first flying
            then landing;

        transition landing_To_off
            first landing
            then off;

        transition flying_To_fault
            first flying
            then fault;

        transition fault_To_off
            first fault
            then off;
    }

    // the battery is a referential part of the quadcopter
    ref part battery [1]{
        attribute capacity :> ISQ::electricCharge;
        attribute voltage :> ISQ::electricPotential;
        attribute power :> ISQ::power;
        port dcPwrOutPort;
        perform produceControlledFlight.provideElectricalPower;
    }

    // composite parts of the quadcopter
    part powerDistributionBoard{
        port dcPwrInPort;
        port propulsionPwrOutPort;
        port controlPwrOutPort;
        perform produceControlledFlight.distributeElectricalPower;
    }

    part receiver{
        port cmdInPort;
        port desiredMotionOutPort;
        perform produceControlledFlight.receiveFlightCommand;
    }

    part flightController{
        attribute controllerRate : Real;
        port desiredMotionInPort;
        port sensedStateInPort;
        port controlPwrInPort;
        port motorCmdOutPort;
        port telemetryOutPort;
        perform produceControlledFlight.computeFlightControl;
    }

    part sensorSuite{
        port controlPwrInPort;
        port sensedStateOutPort;
        perform produceControlledFlight.measureVehicleState;

        part imu{
            attribute sampleRate : Real;
            port attitudeOutPort;
        }

        part barometer{
            attribute altitudeEstimate :> ISQ::length;
            port altitudeOutPort;
        }

        part gps{
```

```
                attribute positionAccuracy :> ISQ::length;
                port positionOutPort;
            }
        }

        // four electronic speed controllers
        part esc [4]{
            port motorCmdInPort;
            port propulsionPwrInPort;
            port motorPwrOutPort;
            perform produceControlledFlight.modulateMotorPower;
        }

        // four motors
        part motor [4]{
            attribute maxPower :> ISQ::power;
            attribute rotationalSpeed : Real;
            port motorPwrInPort;
            port shaftOutPort;
        }

        // four propellers generate rotor thrust
        part propeller [4]{
            attribute diameter :> ISQ::length;
            attribute pitch :> ISQ::length;
            port shaftInPort;
            port rotorThrustOutPort;
            perform produceControlledFlight.generateRotorThrust;
        }

        part thrustMixer{
            port rotorThrustInPort;
            port thrustOutPort;
            perform produceControlledFlight.combineThrust;
        }

        part telemetryRadio{
            port telemetryInPort;
            port telemetryOutPort;
            perform produceControlledFlight.reportTelemetry;
        }

        part frameBody{
            part centerBody;
            part arm [4];
            part landingGear;
            part payloadMount;
            part propellerGuard [4];
        }

        // binding connections assert that external quadcopter ports
        // are the same as selected internal subsystem ports
        bind cmdPort = receiver.cmdInPort;
        bind telemetryPort = telemetryRadio.telemetryOutPort;
        bind thrustOutPort = thrustMixer.thrustOutPort;

        // power connections
        connect battery.dcPwrOutPort to powerDistributionBoard.dcPwrInPort;
        connect powerDistributionBoard.controlPwrOutPort to sensorSuite.controlPwrInPort;
        connect powerDistributionBoard.controlPwrOutPort to flightController.controlPwrInPort;
        connect powerDistributionBoard.propulsionPwrOutPort to esc.propulsionPwrInPort;

        // command and control connections
        connect receiver.desiredMotionOutPort to flightController.desiredMotionInPort;
        connect sensorSuite.sensedStateOutPort to flightController.sensedStateInPort;
        connect flightController.motorCmdOutPort to esc.motorCmdInPort;

        // propulsion chain connections
        connect esc.motorPwrOutPort to motor.motorPwrInPort;
        connect motor.shaftOutPort to propeller.shaftInPort;
        connect propeller.rotorThrustOutPort to thrustMixer.rotorThrustInPort;

        // telemetry connections
        connect flightController.telemetryOutPort to telemetryRadio.telemetryInPort;
    }
}

// create a RequirementsAllocation package that contains the allocation of
// requirements to the quadcopter design features
package RequirementsAllocation{
```

```
        public import Requirements::*;
        public import PartsTree::*;

        allocate quadcopterSpecification to quadcopter{
            allocate quadcopterSpecification.physical.weight to quadcopter.mass;
            allocate quadcopterSpecification.physical.size to quadcopter.diagonalSize;
            allocate quadcopterSpecification.flightPerformance.flightTime to quadcopter.flightTime;
            allocate quadcopterSpecification.flightPerformance.payload to quadcopter.payloadMass;
            allocate quadcopterSpecification.flightPerformance.hover to quadcopter.hoverStability;

            allocate quadcopterSpecification.userInterface.commandInput to quadcopter.cmdPort;
            allocate quadcopterSpecification.userInterface.telemetry to quadcopter.telemetryPort;

            allocate quadcopterSpecification.energy.batteryVoltage to quadcopter.battery.voltage;
            allocate quadcopterSpecification.energy.powerDistribution to quadcopter.
    powerDistributionBoard;

            allocate quadcopterSpecification.sensingAndControl.attitudeSensing to quadcopter.
    sensorSuite.imu;
            allocate quadcopterSpecification.sensingAndControl.altitudeSensing to quadcopter.
    sensorSuite.barometer;
            allocate quadcopterSpecification.sensingAndControl.flightControl to quadcopter.
    flightController;

            allocate quadcopterSpecification.safety.failsafe to quadcopter.quadcopterStates.fault;
            allocate quadcopterSpecification.safety.lowBatteryProtection to quadcopter.
    batteryStateOfCharge;
            allocate quadcopterSpecification.safety.propellerGuarding to quadcopter.frameBody.
    propellerGuard;
        }
    }
}
```

Listing B.1: Toy quadcopter SysML-style model

Appendix C

Domain-Agnostic FMEA Ontology

This appendix describes the domain-agnostic Failure Mode and Effects Analysis (FMEA) ontology used in this thesis. The ontology is represented as a JSON-based graph structure and is used as a semantic reasoning input for the AI-supported failure analysis component described in Chapter 3. Its purpose is to provide the AI component with a reusable reasoning structure for FMEA, while the parsed SysML model provides the system-specific context.

The ontology should not be interpreted as a complete formal ontology in the strict OWL sense. Instead, it is a GraphRAG-ready ontology representation. It contains concepts, relationships, reasoning rules, question templates, and standards-based references that can be retrieved and supplied to the AI during FMEA generation, review, explanation, and revision.

C.1 Purpose of the Ontology

The ontology was created to support domain-agnostic FMEA reasoning. This means that it is not specific to the flashlight model, the quadcopter model, or any single engineering domain. Instead, it provides general concepts and reasoning structures that can be applied across mechanical, electrical, electronic, software, control, thermal, fluidic, chemical, material, human, process, interface, environmental, cybersecurity, data/AI, maintenance, and supply-chain failure domains.

The ontology has three main purposes in the methodology:

1. to guide AI-supported generation of candidate failure modes;
2. to support review of generated FMEA rows by checking conceptual consistency;
3. to provide a shared semantic structure for explanation, elicitation, and revision.

In the proposed framework, the ontology works in parallel with the model parser. The parser provides system-specific context extracted from the SysML model. The ontology provides domain-agnostic reasoning guidance. Therefore, the parser tells the AI what system information to reason about, while the ontology guides how the AI should structure its reasoning.

C.2 Ontology Creation Process

The ontology was created through a four-stage process, shown conceptually in Figure C.1. The process begins with standards and guidance documents, then moves through a human-readable outline, then ontology formalisation, and finally transformation into a GraphRAG-ready representation.

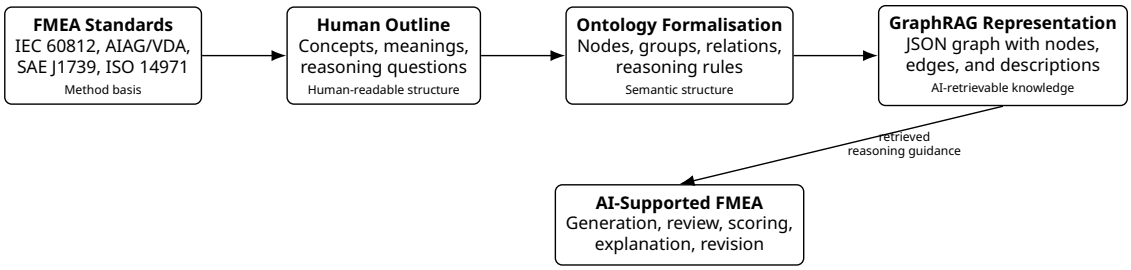


Figure C.1: Ontology creation process from standards-based guidance to a GraphRAG-ready ontology representation.

C.2.1 Stage 1: Standards-Based Concept Extraction

The first stage was to identify the main concepts and reasoning steps used in established FMEA and risk-analysis practice. The ontology uses four standards or guidance sources as its foundation:

- IEC 60812:2018, used as a general FMEA and FMECA method reference;
- AIAG/VDA FMEA Handbook, used for its seven-step FMEA workflow;
- SAE J1739, used for design FMEA, process FMEA, and FMEA-MSR concepts;
- ISO 14971 risk-management thinking, used for hazard, hazardous situation, harm, and risk-control logic.

These sources provide the methodological basis for the ontology. They help identify the required reasoning concepts, such as item, function, failure mode, effect, cause, control, severity, occurrence, detection, recommended action, evidence, and residual risk.

C.2.2 Stage 2: Human-Readable Ontology Outline

The second stage was to translate the standards-based concepts into a human-readable outline. This step was necessary because standards describe FMEA methods, but they do not directly provide an AI-usable ontology. The outline organised the concepts into groups, clarified their meanings, and identified relationships between them.

For example, the outline separates the following ideas:

- an item or component performs a function;
- a function can fail through a failure mode;
- a failure mode may have one or more causes;
- a failure mode produces local, next-higher-level, system, or end-user effects;
- current controls may prevent causes, detect failures, or mitigate effects;
- risk ratings depend on severity, occurrence, and detection;
- recommended actions should reduce risk, uncertainty, or weak traceability.

This human outline provided the bridge between standards-based FMEA knowledge and the structured JSON ontology.

C.2.3 Stage 3: Ontology Formalisation

The third stage was to formalise the outline into a graph-like ontology structure. Concepts were represented as nodes, and their relationships were represented as edges. Each node includes an identifier, label, group, title, and description. Some nodes also include AI question prompts, which are used to guide the AI when information is missing or when deeper reasoning is required.

The formalisation produced concepts across several categories, including FMEA process steps, context concepts, structure concepts, function concepts, failure-logic concepts, risk concepts, rating concepts, control concepts, action concepts, evidence concepts, FMEA types, failure domains, failure-mode categories, cause categories, and AI question templates.

C.2.4 Stage 4: GraphRAG-Ready Transformation

The final stage was to transform the ontology into a GraphRAG-ready JSON representation. In this representation, concepts are stored as retrievable knowledge nodes, and semantic relations are stored as edges. This enables the AI component to retrieve relevant ontology fragments during analysis instead of relying only on its internal model knowledge.

This GraphRAG-ready form supports AI reasoning in three ways:

1. it makes relevant concepts retrievable;
2. it makes relationships between concepts explicit;
3. it provides reasoning rules and question templates that can be injected into AI prompts.

This is important for model integrity because it reduces unbounded AI reasoning. The AI is guided by explicit concepts and relations rather than being asked to invent an FMEA structure independently.

C.3 JSON Structure of the Ontology

The ontology JSON contains five main top-level elements: name, description, references, reasoning rules, nodes, and edges. Table C.1 summarises these elements.

Table C.1: Top-level JSON structure of the domain-agnostic FMEA ontology

Element	Type	Purpose
`name`	String	Provides the name of the ontology.
`description`	String	Describes the purpose and domain coverage of the ontology.
`references`	Array	Lists standards and guidance sources used to ground the ontology.
`reasoning_rules`	Array	Defines rules that guide AI-supported FMEA reasoning.
`nodes`	Array	Contains ontology concepts, grouped by semantic category.
`edges`	Array	Contains graph relationships between ontology concepts.

The current ontology contains 4 references, 10 reasoning rules, 168 nodes, and 355 edges. Table C.2 summarises the size of the ontology artefact.

Table C.2: Size of the ontology artefact

Ontology element	Count
References	4
Reasoning rules	10
Nodes	168
Edges	355

C.4 Standards Foundation

The ontology is grounded in established FMEA and risk-analysis sources. These sources are not used as direct executable rules, but as methodological anchors for the ontology. Table C.3 summarises the four reference sources represented in the JSON file.

Table C.3: Standards and guidance sources represented in the ontology

Source	Role in the ontology
IEC 60812:2018	Provides the general FMEA/FMECA method basis for planning, performing, documenting, maintaining, and identifying how items or processes fail to perform functions.
AIAG/VDA FMEA Handbook	Provides the seven-step FMEA workflow: planning and preparation, structure analysis, function analysis, failure analysis, risk analysis, optimisation, and results documentation.
SAE J1739	Provides concepts relevant to Design FMEA, Process FMEA, and monitoring/system-response FMEA.
ISO 14971 risk-management thinking	Provides concepts for connecting technical failure modes to hazards, hazardous situations, harm, and risk controls.

C.5 Reasoning Rules

The ontology includes ten reasoning rules. These rules are not ordinary FMEA table columns. Instead, they guide the AI in how to reason when generating or reviewing FMEA content. Table C.4 summarises the rules.

Table C.4: Reasoning rules included in the ontology

Rule ID	Name	Purpose
rule-01	Function-first reasoning	Before proposing failure modes, identify the intended function, operating mode, environment, input, output, performance parameter, and requirement.
rule-02	Separate mode, cause, and effect	Prevents the AI from mixing what goes wrong, why it happens, and what happens because of it.
rule-03	Multiple levels of effect	Encourages local, next-higher-level, system, end-user, safety, regulatory, or business effects where relevant.

Rule ID	Name	Purpose
rule-04	Cross-domain sweep	Forces consideration across mechanical, electrical, software, control, thermal, interface, human, environmental, cybersecurity, data/AI, maintenance, and supply-chain domains.
rule-05	Interface suspicion	Treats interfaces as high-risk zones because mismatched assumptions, timing, units, tolerances, protocols, and responsibilities often create hidden failures.
rule-06	Evidence over confidence	Requires ratings and claims about controls to be supported by evidence such as tests, simulations, inspections, process capability, supplier records, field data, or expert rationale.
rule-07	Current-control specificity	Requires current controls to specify whether they prevent causes, detect failures, mitigate effects, monitor diagnostics, or trigger responses.
rule-08	Action target	Requires recommended actions to explicitly target severity, occurrence, detection, diagnostic coverage, uncertainty, or traceability.
rule-09	Residual risk	Requires reassessment after actions and records remaining risk with acceptance rationale.
rule-10	AI follow-up	Requires targeted follow-up questions when key context is missing, and explicit assumption marking when the analysis must proceed with incomplete information.

These reasoning rules are central to the use of the ontology. They reduce the risk of unsupported or vague AI-generated FMEA rows by forcing the reasoning process to remain function-based, evidence-aware, and reviewable.

C.6 Node Groups

The ontology contains 168 nodes organised into conceptual groups. Each node represents a concept that may be used during FMEA reasoning. The node groups are summarised in Table C.5.

Table C.5: Main node groups in the domain-agnostic FMEA ontology

Node group	Count	Purpose
Control Method	18	Represents methods such as design review, simulation, inspection, redundancy, derating, fail-safe states, maintenance plans, and change control.

Node group	Count	Purpose
Failure Domain	17	Represents broad domains where failures may arise, including mechanical, electrical, software, control, thermal, process, interface, cybersecurity, data/AI, maintenance, and supply chain.
Cause Category	13	Represents typical classes of causes, such as requirement ambiguity, design weakness, manufacturing defect, assembly error, contamination, overload, supplier variation, misuse, and software defect.
Failure Mode Category	12	Represents generic patterns of functional deviation, such as loss of function, partial function, degraded function, intermittent function, unintended function, wrong function, early function, late function, unstable function, out-of-tolerance, leakage, and blockage.
AI Question Template	12	Provides question prompts for context, function, interface, failure mode, effect, cause, control, evidence, missing information, cross-domain review, and prioritisation.
FMEA Type	9	Represents variants such as System FMEA, Design FMEA, Process FMEA, FMEA-MSR, Software FMEA, Hardware FMEA, FMECA, FMEDA, and Human Factors FMEA.
Context	8	Represents scope, boundary, environment, lifecycle phase, operating mode, use scenario, stakeholder, and assumption.
Failure Logic	8	Represents failure mode, failure effect, failure cause, root cause, mechanism, local effect, next-higher-level effect, and end-user effect.
FMEA Step	7	Represents the workflow steps from planning and preparation to results documentation.
Structure	7	Represents item, system, subsystem, component, interface, input, and output.
Function	6	Represents function, primary function, secondary function, safety function, diagnostic function, and functional chain.
Rating	6	Represents severity, occurrence, detection rating, RPN, action priority, and criticality.
Risk Concept	5	Represents hazard, hazardous situation, harm, risk, and residual risk.
Control	4	Represents current control, prevention control, detection control, and diagnostic monitoring.
Action	3	Represents recommended action, action owner, and action deadline.
Standard	4	Represents the standards and guidance sources used to ground the ontology.
Other specialised groups	31	Include requirement, metric, decision rule, evidence, knowledge, and specific mechanical, electrical, software, thermal, data, AI, and interface failure-mode concepts.

The node groups show that the ontology is broader than a simple FMEA table schema. It

includes not only output fields, but also reasoning steps, failure domains, cause categories, control methods, rating logic, and AI question prompts.

C.7 FMEA Process Concepts

The ontology includes a process structure aligned with a staged FMEA workflow. The main FMEA step nodes are:

1. planning and preparation;
2. structure analysis;
3. function analysis;
4. failure analysis;
5. risk analysis;
6. optimisation;
7. results documentation.

These steps are linked through precedence relations. This means that the ontology does not only describe FMEA concepts; it also represents an expected reasoning sequence. Planning and preparation define the scope, boundary, assumptions, stakeholders, lifecycle phase, and risk acceptance criteria. Structure analysis identifies the item, system, subsystem, component, interface, and environment. Function analysis identifies functions, requirements, inputs, outputs, constraints, operating modes, and performance parameters. Failure analysis identifies failure modes, effects, causes, mechanisms, hazards, and harms. Risk analysis evaluates severity, occurrence, detection, RPN, action priority, criticality, and risk. Optimisation improves actions and controls. Results documentation captures evidence, traceability, residual risk, and lessons learned.

This process structure is useful for AI-supported analysis because it discourages the AI from jumping directly to failure modes without first considering structure and function.

C.8 Failure Logic Concepts

The ontology contains a detailed failure-logic structure. At the centre is the relationship between function, failure mode, cause, effect, and control. The key idea is that failure modes should be derived from functions. A failure mode is not simply a bad event; it is a deviation from what the item or function is supposed to do.

The ontology distinguishes between:

- **failure mode**: the way the function fails;
- **failure cause**: the reason the failure mode occurs;
- **failure mechanism**: the physical, logical, chemical, human, cyber, or process mechanism behind the cause;
- **failure effect**: the consequence of the failure mode;
- **local effect**: the consequence on the same item or process step;
- **next-higher-level effect**: the consequence on a subsystem or downstream function;

- **end-user effect**: the consequence experienced by the user, operator, maintainer, customer, or other stakeholder.

This distinction is important because AI-generated FMEA rows often risk mixing mode, cause, and effect. The ontology provides a conceptual structure that helps prevent this confusion.

C.9 Failure Domains and Failure Mode Categories

The ontology includes a cross-domain failure sweep. This is useful because failures can arise from many domains, not only from physical component breakage. The failure domain nodes include mechanical, electrical, electronic, software, control, thermal, fluidic, chemical, material, human, process, interface, environmental, cybersecurity, data/AI, maintenance, and supply-chain failures.

The ontology also includes generic failure mode categories. These categories help transform functions into possible deviations. Examples include:

- loss of function;
- partial function;
- degraded function;
- intermittent function;
- unintended function;
- wrong function;
- early function;
- late function;
- unstable or oscillatory function;
- out-of-tolerance output;
- leakage;
- blockage.

These categories support systematic failure-mode generation. Instead of asking the AI to guess random failures, the ontology guides the AI to ask how each function might be absent, partial, degraded, intermittent, unintended, wrong, early, late, unstable, or outside tolerance.

C.10 Risk, Rating, and Control Concepts

The ontology includes concepts for risk analysis and prioritisation. These include severity, occurrence, detection rating, RPN, action priority, criticality, risk, residual risk, and risk acceptance criteria. These concepts help the AI distinguish between identifying a failure and prioritising it.

Severity is associated with the seriousness of the effect. Occurrence is associated with the likelihood or frequency of the cause or failure mode. Detection rating is associated with the ability of existing controls to detect the cause or failure before the effect reaches the user. RPN is represented as the traditional product of severity, occurrence, and detection, while action priority and criticality are included as additional prioritisation concepts.

The ontology also distinguishes between different types of controls:

- current controls;
- prevention controls;
- detection controls;
- diagnostic monitoring;
- recommended actions.

This distinction supports more precise FMEA rows. For example, a control should not be written vaguely as “testing” without explaining whether it prevents the cause, detects the failure, mitigates the effect, or monitors the system during operation.

C.11 Evidence, Traceability, and Documentation Concepts

A major strength of the ontology is that it includes evidence and traceability concepts. Verification evidence is represented as proof that an action or control works. Examples include test reports, simulation results, inspection data, requirement traces, field data, audit records, or expert rationale.

The ontology also includes traceability as a control method. Traceability links requirements, functions, design elements, tests, risks, actions, and evidence. In the context of this thesis, this is directly connected to model integrity. An FMEA row is more useful when it can be connected back to the model information that supports it.

The ontology therefore supports evidence-aware AI reasoning. It encourages the AI not only to generate plausible failure modes, but also to consider what evidence or rationale supports the generated row.

C.12 AI Question Templates

The ontology includes a set of AI question templates. These templates are designed to guide the AI when context is missing, when reasoning needs to be expanded, or when the user asks for explanation. The question templates include:

- context questions;
- function questions;
- interface questions;
- failure mode questions;
- effect questions;
- cause questions;
- control questions;
- evidence questions;
- counterfactual questions;
- cross-domain questions;
- missing-information questions;
- prioritisation questions.

These templates are useful because they make the ontology interactive. The ontology does not only store concepts; it also suggests what questions should be asked to improve the analysis. This is especially relevant for the chat-based elicitation and revision interface described in Chapter 3.

C.13 Graph Structure and Edge Relations

The ontology contains 355 edges. These edges connect the ontology concepts into a reasoning graph. The edges describe relationships such as:

- standards inform the ontology;
- the ontology enables the AI FMEA facilitator;
- an FMEA study contains FMEA records;
- FMEA process steps precede one another;
- planning defines scope, boundary, assumptions, stakeholders, lifecycle phase, and risk criteria;
- structure analysis identifies items, systems, subsystems, components, interfaces, and environments;
- function analysis identifies functions, requirements, inputs, outputs, constraints, operating modes, and performance parameters;
- failure analysis identifies failure modes, effects, causes, mechanisms, hazards, and harms;
- risk analysis evaluates severity, occurrence, detection, RPN, action priority, criticality, and risk;
- optimisation improves recommended actions, prevention controls, detection controls, diagnostic monitoring, and residual risk;
- results documentation captures verification evidence, traceability, and lessons learned.

The graph structure is important because it allows the ontology to be used for retrieval. For example, if the AI is generating failure modes for an interface, the graph can retrieve interface-related failure questions, interface failure domains, possible causes, detection controls, and evidence questions. This makes the ontology suitable for GraphRAG.

C.14 Use of the Ontology in the Proposed Framework

Within the methodology, the ontology is used together with the parsed model context. The parsed model context is system-specific. It describes parts, actions, ports, flows, connections, requirements, allocations, and state/control information extracted from the SysML model. The ontology is domain-agnostic. It describes the general structure of FMEA reasoning.

The AI component receives both inputs. The parser output grounds the AI in the specific system. The ontology guides the AI in structuring the analysis. This combination supports several activities:

1. **Generation**: the ontology guides the AI to derive failure modes from functions and model context.

2. **Review**: the ontology helps check whether generated rows correctly separate item, function, failure mode, cause, effect, control, and evidence.
3. **Scoring**: the ontology provides concepts for severity, occurrence, detection, RPN, action priority, and residual risk.
4. **Explanation**: the ontology provides a shared vocabulary for explaining why a row was generated.
5. **Revision**: the ontology helps preserve FMEA structure when the user asks for changes.

C.15 Limitations of the Ontology

The ontology has several limitations. First, it is not a complete formal ontology in a description-logic sense. It does not include full formal constraints, inference rules, or machine-verifiable consistency conditions. It is better understood as a structured GraphRAG knowledge artefact.

Second, some node descriptions are written in informal language. This can be useful for internal AI prompting, but thesis-facing or industrial versions should use more neutral wording. Informal phrases should be revised before the ontology is presented as a formal engineering artefact.

Third, the ontology is domain-agnostic. This is useful for reuse, but it also means that it does not contain detailed domain-specific failure physics for any one system. For example, it can guide reasoning about electrical failure modes, but it does not replace detailed electrical reliability data, simulation results, or expert domain knowledge.

Fourth, the ontology depends on governance and maintenance. If new FMEA practices, standards, failure domains, or project-specific concepts are introduced, the ontology must be updated. Otherwise, it may become stale in the same way that ordinary engineering documents can become stale.

Finally, the ontology does not remove the need for human review. It improves the structure and consistency of AI-supported reasoning, but it does not guarantee correctness. Engineering judgement remains necessary for validating generated failure modes, risk scores, recommended actions, and evidence claims.

C.16 Summary

The domain-agnostic FMEA ontology is a central semantic artefact in the methodology. It provides the AI component with a structured reasoning frame for FMEA. The ontology is built from standards-based FMEA concepts, translated into a human-readable outline, formalised into graph-like nodes and edges, and transformed into a GraphRAG-ready JSON representation.

The ontology contains standards references, reasoning rules, concept nodes, and semantic edges. Its concepts cover FMEA steps, context, structure, function, failure logic, risk, ratings, controls, actions, evidence, FMEA types, failure domains, cause categories, failure-mode categories, and AI question templates. Its edges connect these concepts into a reasoning graph that supports generation, review, explanation, scoring, and revision.

In the proposed framework, the ontology complements the model parser. The parser provides system-specific context, while the ontology provides domain-agnostic reasoning guidance. Together, they support more bounded, traceable, consistent, and reviewable AI-supported failure analysis.